Lecture 9: Latent Diffusion Model and Stable Diffusion

잠재 공간 디퓨전 — 모든 구성 요소의 결합

목차 (Table of Contents)

- 0. 개요 (Overview)
  - 0.1 논문 정보
  - 0.2 선수 지식 (Prerequisites)
  - 0.3 학습 목표 (Learning Objectives)
  - 0.4 구성 요소 한눈에 보기
- 1. 동기: 왜 잠재 공간에서 디퓨전하는가?
  - 1.1 픽셀 공간 디퓨전의 문제
  - 1.2 핵심 통찰
- 2. 오토인코더 (Stage 1)
  - 2.1 KL-VAE vs VQ-GAN
  - 2.2 압축률 (Downsampling Factor)
  - 2.3 Scaling Factor
  - 2.4 PyTorch 구현: 간소화된 KL-VAE Encoder/Decoder
- 3. 잠재 공간 디퓨전 (Stage 2)
  - 3.1 학습
  - 3.2 생성
  - 3.3 U-Net 구조 (SD 1.x/2.x)
  - 3.4 PyTorch 구현: Latent Diffusion 학습 루프
- 4. 전체 파이프라인 정리
  - 4.1 학습 단계
  - 4.2 생성 단계
  - 4.3 PyTorch 구현: CFG를 포함한 전체 생성 파이프라인
- 5. Stable Diffusion 버전별 진화
  - 5.1 SD 1.x (2022.08)
  - 5.2 SD 2.x (2022.11)
  - 5.3 SDXL (2023.07)
  - 5.4 SD 3 (2024.02)
  - 5.5 FLUX (2024.08)
- 6. 조건부 제어 확장
  - 6.1 LDM의 유연한 조건부 메커니즘
  - 6.2 Image-to-Image (img2img)
  - 6.3 Inpainting
  - 6.4 PyTorch 구현: Image-to-Image (img2img)
- 7. 실용적 고려사항
  - 7.1 샘플러 선택
  - 7.2 메모리 최적화
  - 7.3 Fine-tuning 방법
  - 7.4 PyTorch 구현: 간소화된 LoRA Fine-tuning
- 8. 핵심 수식 요약

- 9. 연결 지도: 강의 1~9의 통합
- 10. 다음 강의 미리보기
- 참고문헌 (References)

---

0. 개요 (Overview)

0.1 논문 정보

> 📄 Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models", **CVPR 2022**
>
> 저자: Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer
>
> (VQ-GAN의 Esser, Ommer와 동일 연구팀)

> 💡 **왜 이 논문이 중요한가?** 2022년 이전까지 디퓨전 모델(DDPM, Score-based 등)은 이론적으로 뛰어나지만 **현실적으로 사용하기 어려웠다**. 256×256 해상도에서도 학습에 V100 수백 GPU-day가 필요했고, 512×512는 사실상 연구실 수준에서 불가능했다. 이 논문은 "디퓨전을 잠재 공간에서 하면 된다"는 단순하지만 강력한 아이디어로 **계산 비용을 수십 배 줄이면서 품질은 유지하는 방법**을 제시했다. 이후 Stable Diffusion, DALL-E 3, Midjourney 등 사실상 모든 상용 이미지 생성 모델이 이 패러다임을 채택했다. 한마디로, **현대 이미지 생성 AI의 표준 아키텍처를 확립한 논문**이다.

0.2 선수 지식 (Prerequisites)

이 강의를 이해하기 위해 다음 개념들에 대한 이해가 필요하다:

선수 지식	강의	핵심 개념
VQ-GAN / VAE	강의 4	이미지를 잠재 공간으로 압축하는 오토인코더, 코드북 양자화, 지각적 손실
DDPM	강의 7	Forward/Reverse 디퓨전 과정, ϵ-prediction, 노이즈 스케줄
CLIP + CFG	강의 8	텍스트-이미지 대조 학습, Classifier-Free Guidance로 조건 강화

⚠️ **중요**: LDM은 위 세 가지를 모두 결합한 모델이므로, 각 구성 요소를 독립적으로 이해한 뒤에 이 강의를 학습하는 것을 권장한다.

0.3 학습 목표 (Learning Objectives)

이 강의를 마치면 다음을 할 수 있어야 한다:

1. 픽셀 공간 디퓨전 대비 **잠재 공간 디퓨전의 장점**을 설명할 수 있다
2. LDM의 **2-Stage 학습 구조** (오토인코더 → 디퓨전)를 이해하고 설명할 수 있다
3. Stable Diffusion의 **전체 파이프라인** (인코딩 → 노이즈 → 디노이징 → 디코딩)을 코드로 구현할 수 있다
4. **img2img**, **inpainting** 등 응용 기법의 원리를 이해할 수 있다
5. SD 1.x → 2.x → XL → 3 → FLUX까지의 **기술 진화 흐름**을 파악할 수 있다
6. **LoRA** 등의 효율적 fine-tuning 기법을 적용할 수 있다

0.4 구성 요소 한눈에 보기

Latent Diffusion Model(LDM)은 이전 강의에서 배운 모든 것을 하나로 결합한 모델이다:

구성 요소	출처	역할
VQ-GAN / KL-VAE	강의 4	이미지 ↔ 잠재 공간 변환
DDPM	강의 7	잠재 공간에서 디퓨전

CLIP	강의 8	텍스트 조건 인코딩
CFG	강의 8	조건 강화

> 💡 **왜 이렇게 여러 구성 요소를 결합하는가?** 각 구성 요소는 **서로 다른 문제를 해결한다**. 오토인코더는 "고차원 이미지를 효율적으로 표현하는 방법"을 해결하고, DDPM은 "복잡한 데이터 분포를 점진적으로 학습하는 방법"을 해결하며, CLIP은 "텍스트와 이미지의 의미적 연결"을 해결하고, CFG는 "조건 신호를 강화하여 원하는 이미지를 얻는 방법"을 해결한다. LDM의 핵심 기여는 이 독립적인 해결책들이 **어떤 순서로, 어떤 인터페이스를 통해 연결되어야 하는지**를 제시한 것이다. 즉, LDM은 새로운 구성 요소를 발명한 것이 아니라, 기존 구성 요소들의 **최적 조합 방법**을 발견한 모델이다.

각 구성 요소 사이의 **정보 흐름**을 정확히 이해하는 것이 중요하다:

```
[학습 시 정보 흐름]
이미지 x ──→ [VAE Encoder] ──→ z₀ (잠재 표현)
                                  ↓
텍스트 y ──→ [CLIP Encoder] ──→ c (조건 임베딩) ──→ Cross-Attention으로 U-Net에 주입
                                                          ↓
                              z_t = noise(z₀, t) ──→ [U-Net] ──→ ε^(노이즈 예측)
                                                          ↓
                                                    Loss = ‖ε - ε‖²

[생성 시 정보 흐름]
텍스트 y ──→ [CLIP Encoder] ──→ c
                                  ↓
z_T ~ N(0,I) ──→ [U-Net + CFG] × T steps ──→ z₀ ──→ [VAE Decoder] ──→ 이미지 x
```

여기서 핵심적인 설계 원칙은 **관심사의 분리(Separation of Concerns)**이다. VAE는 "어떻게 압축할 것인가"만 담당하고, U-Net은 "어떻게 디노이징할 것인가"만 담당하며, CLIP은 "텍스트를 어떻게 이해할 것인가"만 담당한다. 이 분리 덕분에 각 구성 요소를 **독립적으로 개선하거나 교체**할 수 있다. 실제로 SD 1.x → 2.x에서는 CLIP만 교체했고, SD 3에서는 U-Net을 DiT로 교체했다.

---

1. 동기: 왜 잠재 공간에서 디퓨전하는가?

> 💡 **왜 이 질문이 중요한가?** "잠재 공간에서 디퓨전한다"는 아이디어는 단순해 보이지만, 이것이 **왜 작동하는지**를 깊이 이해하지 않으면 LDM의 설계 결정(압축률 선택, KL 가중치, scaling factor 등)이 왜 그런 값인지 납득할 수 없다. 이 섹션에서는 "잠재 공간 디퓨전"이라는 아이디어의 **이론적 근거**를 정보 이론 관점에서 설명한다.

1.1 픽셀 공간 디퓨전의 문제

DDPM을 256×256 RGB 이미지에 직접 적용하면:

$$\text{디퓨전 공간 차원} = 256 \times 256 \times 3 = 196{,}608$$

- U-Net이 196,608차원의 입출력을 처리 → **메모리와 계산량이 막대**
- 512×512 이상은 사실상 불가능
- 고해상도 이미지의 대부분은 **저수준 디테일**(질감, 패턴)이며, 의미적 구조는 훨씬 적은 차원으로 표현 가능

구체적인 계산 비용 비교:

512×512×3 해상도의 경우를 구체적으로 살펴보자:

공간	차원 수	U-Net 1회 forward (FLOPs 추정)	50 step 생성 시 총 비용

픽셀 공간 512×512×3	786,432	~500 GFLOPs	~25 TFLOPs
잠재 공간 64×64×4	16,384	~10 GFLOPs	~0.5 TFLOPs
비율	**48배 축소**	**~50배 절약**	**~50배 절약**

여기서 계산량이 차원 수에 단순히 비례하지 않고 오히려 더 큰 비율로 줄어드는 이유는, U-Net 내부의 Self-Attention 연산이 공간 크기의 **제곱**에 비례하기 때문이다. 즉, 공간이 64×64일 때의 Self-Attention 비용은 512×512일 때의 $(64^2/512^2)^2 \approx 1/4096$이 된다.

> **"그러면 이미지를 그냥 작게 리사이즈하면 되지 않나?"** 이 질문은 매우 자연스럽지만, 답은 **아니오**이다. 단순 다운스케일링(bilinear/bicubic interpolation)은 고주파 정보(sharp edge, texture, fine detail)를 **비가역적으로 파괴**한다. 512×512 이미지를 64×64로 리사이즈하면, 머리카락 한 올, 눈동자의 반사광 같은 디테일은 완전히 사라지고 복원할 수 없다. 반면 **학습된 오토인코더**는 고주파 정보를 **잠재 벡터의 특정 차원에 인코딩**하여 보존한다. 디코더는 이 잠재 벡터로부터 고주파 디테일을 **재구성(hallucinate)** 할 수 있다. 이것이 가능한 이유는 오토인코더가 학습 데이터의 통계적 규칙성(예: "피부 질감은 이런 패턴", "나뭇잎은 이런 프랙탈 구조")을 학습했기 때문이다.

1.2 핵심 통찰

> "이미지의 의미적 내용과 지각적 세부사항은 분리할 수 있다."

이 통찰의 이론적 배경은 **정보 병목 원리(Information Bottleneck Principle)**이다. 자연 이미지에서 인접 픽셀은 극도로 높은 상관관계를 가진다. 예를 들어, 하늘의 한 픽셀이 파란색이면 옆 픽셀도 거의 확실히 파란색이다. 이는 786,432차원(512×512×3) 중 대부분이 **중복 정보(redundant information)**임을 의미한다.

정보 이론적으로 표현하면, 자연 이미지의 **내재 차원(intrinsic dimensionality)**은 픽셀 차원보다 훨씬 낮다:

$$\dim_{\text{intrinsic}}(\mathcal{X}_{\text{natural images}}) \ll \dim_{\text{pixel}}(\mathcal{X}) = H \times W \times C$$

좋은 오토인코더는 이 내재 차원에 가까운 잠재 공간을 학습한다. 즉, **의미적으로 중요한 정보**(물체의 형태, 색상 분포, 공간 배치)는 보존하고, **지각적으로 중복된 정보**(인접 픽셀 간 상관관계, 반복적 텍스처 패턴)는 제거한다.

1. **오토인코더**(VQ-GAN/VAE)로 이미지를 **저차원 잠재 공간**으로 압축
2. 잠재 공간에서 디퓨전 수행 → **48배 차원 축소**
3. 디코더로 잠재 표현을 이미지로 복원

```
이미지 256×256×3 → [Encoder] → 잠재 표현 32×32×4 → [Decoder] → 이미지 256×256×3
    196,608차원                   4,096차원                      196,608차원
                                (48배 축소)
```

이것이 작동하는 직관적 이유:

디퓨전 모델의 역할은 "데이터 분포 $p(x)$를 학습하는 것"이다. 픽셀 공간에서 이 분포를 학습하려면, U-Net은 동시에 두 가지를 배워야 한다:

1. **의미적 구조**: "고양이의 눈은 이 위치에 있어야 한다" (고수준)
2. **지각적 디테일**: "이 픽셀의 RGB 값은 (142, 87, 53)이어야 한다" (저수준)

이 두 과제를 하나의 네트워크가 동시에 해결하는 것은 매우 어렵다. LDM은 이를 **분업**한다:

- **오토인코더**: 지각적 디테일 ↔ 잠재 표현 변환을 담당 (한 번만 학습)
- **디퓨전 모델**: 잠재 공간에서 의미적 구조만 학습 (핵심 학습)

이 분업의 핵심 이점은, 디퓨전 모델이 "피부의 미세한 질감"이나 "나뭇잎의 정확한 색상 그라데이션"을 학습할 필요 없이, "얼굴이 여기에 있고 나무가 저기에 있다"는 **구조적 정보만 학습하면 된다**는 것이다. 디테일 복원은 디코더가 알아서 처리한다.

⚠️ **핵심 포인트**: 잠재 공간에서의 디퓨전은 의미적으로 중요한 정보에 집중하면서, 고주파 디테일은 디코더에게 맡기는 전략이다. 이로 인해 학습 속도가 크게 향상되고, GPU 메모리 요구량이 대폭 감소한다.

> 💡 **잠재 공간 디퓨전의 숨겨진 장점: 더 쉬운 분포 학습** 픽셀 공간의 이미지 분포 $p(x)$는 매우 복잡하다 — 유효한 이미지는 전체 픽셀 공간의 극히 작은 부분(매니폴드)에만 존재한다. 반면, 잘 학습된 오토인코더의 잠재 공간 분포 $p(z)$는 훨씬 **매끄럽고 규칙적**이다 (특히 KL 정규화 덕분에 대략 가우시안에 가깝다). 디퓨전 모델은 가우시안 노이즈에서 시작하므로, 타겟 분포가 이미 가우시안에 가까운 잠재 공간에서 디노이징하는 것이 픽셀 공간보다 **수렴이 빠르고 안정적**이다.

---

2. 오토인코더 (Stage 1)

> 💡 **왜 오토인코더를 별도로 학습하는가?** 오토인코더를 디퓨전과 함께 end-to-end로 학습하지 않는 이유는 **안정성과 효율성** 때문이다. 만약 디퓨전 학습 중에 오토인코더도 함께 변하면, 잠재 공간의 의미가 매 스텝마다 바뀌어서 디퓨전 모델이 "움직이는 표적"을 쫓는 셈이 된다. 오토인코더를 먼저 고정하면, 디퓨전 모델은 **안정적인 잠재 공간**에서 학습할 수 있다. 또한 하나의 오토인코더를 학습한 뒤 **여러 다른 디퓨전 모델에 재사용**할 수 있다는 실용적 장점도 있다 (실제로 SD 1.x와 2.x는 동일한 VAE를 공유한다).

2.1 KL-VAE vs VQ-GAN

LDM은 두 가지 오토인코더를 사용할 수 있다:

	KL-VAE	**VQ-GAN**
잠재 공간	**연속** ($z \in \mathbb{R}^{h \times w \times c}$)	이산 (코드북 인덱스)
정규화	KL Divergence	벡터 양자화 + Commitment Loss
Loss	MSE + KL + (Perceptual)	MSE + Commitment + **GAN** + Perceptual
사용	**Stable Diffusion** (KL-reg)	DALL-E, Parti 등

> 💡 **왜 Stable Diffusion은 VQ-GAN이 아닌 KL-VAE를 선택했는가?** 디퓨전 모델은 **연속 공간에서의 가우시안 노이즈 추가/제거** 과정이다. VQ-GAN의 이산 잠재 공간(코드북 인덱스)에서는 "노이즈를 조금 추가한다"는 연산이 자연스럽지 않다 — 이산 값에 연속 노이즈를 더하면 의미가 깨진다. 반면 KL-VAE의 연속 잠재 공간은 가우시안 노이즈와 자연스럽게 호환된다. $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ 이 수식이 의미를 가지려면, z_0이 연속 공간에 있어야 한다.

Stable Diffusion은 **KL-정규화 오토인코더**를 사용한다. 정확히는 "약한 KL 페널티를 적용한 VAE":

$$\mathcal{L}_{\text{AE}} = \underbrace{\|x - D(E(x))\|^2}_{\text{복원}} + \underbrace{\lambda_{\text{KL}} \cdot D_{\text{KL}}(q(z \mid x) \| \mathcal{N}(0, I))}_{\text{약한 정규화}} + \underbrace{\lambda_{\text{LPIPS}} \cdot \text{LPIPS}(x, D(E(x)))}_{\text{지각적 유사도}} + \underbrace{\lambda_{\text{GAN}} \cdot \mathcal{L}_{\text{GAN}}}_{\text{적대적 학습}}$$

각 Loss 항이 왜 필요한지, 없으면 무슨 문제가 생기는지를 하나씩 살펴보자:

항 1: MSE 복원 Loss $\|x - D(E(x))\|^2$

가장 기본적인 항으로, 입력과 출력의 픽셀 단위 차이를 최소화한다. 하지만 MSE만 사용하면 심각한 문제가 발생한다: **blurry reconstruction**. 그 이유는 다음과 같다.

하나의 잠재 벡터 z에 대해 여러 개의 그럴듯한 복원 이미지가 존재할 수 있다 (예: 머리카락의 정확한 위치, 풀잎의 방향). MSE는 모든 가능한 복원의 **평균**을 선호한다:

$$\arg\min_{\hat{x}} \mathbb{E}\left[\|x - \hat{x}\|^2\right] = \mathbb{E}[x]$$

즉, MSE의 최적해는 가능한 이미지들의 **평균**이며, 평균 이미지는 흐릿하다. 이것이 "MSE는 blur를 만든다"의 수학적 이유이다.

항 2: KL Divergence $D_{\mathrm{KL}}(q(z|x)\|\mathcal{N}(0,I))$

> 💡 **왜 KL 정규화가 필요한가?** KL 정규화 없이 오토인코더를 학습하면, 인코더는 복원 품질을 최대화하기 위해 잠재 공간을 **임의로 복잡한 분포**로 만든다 — 특정 영역에 데이터가 밀집하고, 다른 영역에는 "빈 구멍(hole)"이 존재하며, 불연속적인 경계가 생긴다. 이런 불규칙한 잠재 공간에서 디퓨전을 하면, 노이즈 추가 과정에서 데이터가 이 "빈 구멍"으로 밀려나게 되어 디노이징이 올바르게 작동하지 않는다. KL 항은 잠재 분포를 $\mathcal{N}(0,I)$에 가깝게 강제하여, 잠재 공간을 **매끄럽고 연속적으로** 만든다. 이것이 잠재 공간을 "디퓨전 친화적(diffusion-friendly)"으로 만드는 핵심이다.

KL Divergence의 닫힌 형태(closed-form) 해를 유도하면:

$$D_{\mathrm{KL}}(q(z|x)\|\mathcal{N}(0,I)) = -\frac{1}{2}\sum_{j=1}^{d}\left(1+\log\sigma_j^2-\mu_j^2-\sigma_j^2\right)$$

여기서 $q(z|x)=\mathcal{N}(\mu,\mathrm{diag}(\sigma^2))$이고, μ_j와 σ_j는 인코더가 출력하는 mean과 std의 j번째 차원이다. 각 항의 의미:

- μ_j^2: 평균이 0에서 벗어나면 페널티 → 잠재 공간의 중심이 원점
- σ_j^2: 분산이 1에서 벗어나면 페널티 → 잠재 공간의 스케일이 균일
- $\log\sigma_j^2$: 분산이 너무 작아지는(= 잠재 공간이 붕괴하는) 것을 방지

> 💡 **왜 λ_{KL}이 극도로 작은 값($\sim 10^{-6}$)인가?** 일반적인 VAE에서는 $\lambda_{\mathrm{KL}}=1$을 사용하지만, 이러면 잠재 공간이 $\mathcal{N}(0,I)$에 너무 가까워져서 이미지별 고유 정보가 손실된다 — 결과적으로 복원이 흐릿해진다 (이를 **"posterior collapse"** 또는 **"KL vanishing"**이라 한다). LDM에서는 복원 품질이 최우선이므로, KL 항의 가중치를 극도로 낮춰서 (10^{-6}) 잠재 공간에 "아주 약한 규칙성"만 부여한다. 이렇게 하면 잠재 분포가 정확히 $\mathcal{N}(0,I)$은 아니지만, 대략적으로 **규칙적이고 연속적인 분포**가 되어 디퓨전이 잘 작동한다. 이것이 "약한 KL 페널티"라고 부르는 이유이다.

항 3: LPIPS (Learned Perceptual Image Patch Similarity)

- LPIPS (Learned Perceptual Image Patch Similarity): VGG 특징 공간에서의 거리 → 지각적 품질 향상

> 💡 **왜 MSE 대신 Perceptual Loss가 필요한가?** MSE는 픽셀 차이를 **균일하게** 페널티한다 — 1픽셀 이동한 sharp 엣지와 전체적으로 blur한 이미지가 비슷한 MSE 값을 가질 수 있다. 인간의 시각 체계는 이 둘을 매우 다르게 인식하는데, MSE는 이 차이를 포착하지 못한다. LPIPS는 사전학습된 VGG 네트워크의 중간 레이어 특징(feature)을 비교한다:

$$\mathrm{LPIPS}(x,\hat{x}) = \sum_{l} w_l \cdot \|\phi_l(x)-\phi_l(\hat{x})\|^2$$

여기서 ϕ_l은 VGG의 l번째 레이어 출력이고, w_l은 학습된 가중치이다. VGG의 초기 레이어는 엣지와 텍스처를 감지하고, 깊은 레이어는 구조와 의미를 감지하므로, LPIPS는 **인간의 지각과 유사한 방식으로** 이미지 차이를 측정한다. MSE가 "각 픽셀이 같은가?"를 묻는다면, LPIPS는 "구조적으로 비슷해 보이는가?"를 묻는다. 결과적으로 LPIPS를 사용하면 1-2 픽셀의 미세한 이동보다 **구조적 왜곡**에 더 큰 페널티를 부여하여, 더 날카롭고 지각적으로 우수한 복원을 만든다.

항 4: GAN Loss (PatchGAN Discriminator)

- GAN Loss: PatchGAN 판별자 → 날카로운 디테일

> 💡 **왜 Perceptual Loss에 추가로 GAN Loss가 필요한가?** LPIPS는 사전학습된 VGG 특징에 의존하므로, VGG가 포착하지 못하는 **미세한 텍스처 부자연스러움**이 남을 수 있다. PatchGAN 판별자는 이미지의 각 패치(예: 70×70 영역)가 "진짜 같은지"를 판단하여, 복원 이미지에 남아있는 **미세한 blur나 아티팩트**를 추가로 제거한다. 판별자는 학습 과정에서 점점 더 까다로워지므로, 생성 품질이 지속적으로 향상된다. 단, GAN Loss 비중이 너무 높으면 학습이 불안정해지므로, 적절한 λ_{GAN} 밸런싱이 중요하다.

2.2 압축률 (Downsampling Factor)

f	잠재 크기 (256×256 입력)	잠재 차원	압축률	복원 품질
$f = 4$	64×64	$64 \times 64 \times 3 = 12{,}288$	16배	높음
$f = 8$	**32×32**	$32 \times 32 \times 4 = 4{,}096$	**48배**	좋음 (SD 표준)
$f = 16$	16×16	$16 \times 16 \times c$	~192배	낮음

> 💡 **왜 $f = 8$이 최적인가?** 이 선택은 **복원 품질 vs 디퓨전 효율성의 트레이드오프**에서 나온다. LDM 논문(Table 8)에서 실험적으로 검증한 결과:
>
> - $f = 4$: 복원 품질은 거의 완벽하지만, 잠재 공간 차원이 12,288로 여전히 높아서 디퓨전 학습 비용 절감이 제한적이다. 또한 잠재 공간에 **중복 정보가 많이 남아있어**, 디퓨전 모델이 여전히 지각적 디테일 학습에 리소스를 소비한다.
> - $f = 8$: 차원이 4,096으로 충분히 낮으면서, 중요한 의미적 정보를 대부분 보존한다. 복원 시 FID(Frechet Inception Distance)가 $f = 4$와 거의 동일하다.
> - $f = 16$: 차원이 너무 낮아서 **의미적 정보까지 손실**된다. 복잡한 장면의 구조가 뭉개지며, 복원 FID가 크게 증가한다.
>
> 직관적으로, $f = 8$은 "텍스처는 잃지만 형태는 보존하는" 수준의 압축이다. 이는 디퓨전 모델이 학습해야 할 것과 정확히 일치한다.

Stable Diffusion 표준: $f = 8$, 채널 $c = 4$.

$$256 \times 256 \times 3 \xrightarrow{\text{Encoder}} 32 \times 32 \times 4 \xrightarrow{\text{Decoder}} 256 \times 256 \times 3$$

여기서 잠재 채널 수가 3(RGB)이 아닌 **4**인 이유도 주목하자. 입력의 3채널보다 1채널 더 많은 잠재 공간을 사용하면, 인코더가 RGB 정보 외에 추가적인 **구조적 정보**(예: 깊이, 엣지, 주파수 성분)를 별도 채널에 인코딩할 여유를 갖게 된다. 이 여분의 표현 용량이 복원 품질을 향상시킨다.

> **핵심**: 오토인코더는 디퓨전 학습과 **완전히 독립적으로** 먼저 학습한다. 학습 완료 후 인코더/디코더를 **고정(freeze)**하고, 잠재 공간에서의 디퓨전만 학습.

2.3 Scaling Factor

오토인코더의 출력 분포가 $\mathcal{N}(0, 1)$이 아닐 수 있으므로, 학습 전에 scaling factor를 계산:

$$z = \frac{E(x)}{\text{std}(\{E(x_i)\}_{i=1}^{N})} \quad \text{(학습 데이터 전체의 표준편차로 정규화)}$$

Stable Diffusion 1.x의 scaling factor: **0.18215**

> 💡 **왜 Scaling Factor가 필요한가?** KL 가중치가 매우 작기 때문에(10^{-6}), 인코더의 출력 분포는 $\mathcal{N}(0, 1)$과 상당히 다를 수 있다. 실제로 SD 1.x의 VAE 인코더는 표준편차가 약 $1/0.18215 \approx 5.49$인 분포를 출력한다. DDPM의 forward process는 z_0의 분산이 1이라고 암묵적으로 가정하므로 (노이즈 스케줄이 이에 맞춰 설계됨), 이 불일치를 교정하지 않으면 노이즈 추가 과정에서 **신호 대 잡음비(SNR)가 의도와 달라진다**. Scaling factor를 곱해서 z_0의 분산을 1에 가깝게 맞추면, DDPM의 노이즈 스케줄이 의도대로 작동한다.
>
> 구체적으로: scaling factor $s = 0.18215$를 적용하면, $\text{Var}(s \cdot E(x)) = s^2 \cdot \text{Var}(E(x)) \approx 0.18215^2 \times 5.49^2 \approx 1.0$이 되어 분산이 정규화된다.

2.4 PyTorch 구현: 간소화된 KL-VAE Encoder/Decoder

⚠️ **참고**: 아래 코드는 학습 개념 이해를 위한 간소화 버전이다. 실제 Stable Diffusion의 VAE는 더 깊은 ResNet 블록과 Attention 레이어를 포함한다.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class ResBlock(nn.Module):
    """간단한 ResNet 블록: Conv → GroupNorm → SiLU → Conv → Skip Connection"""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, padding=1)
        self.norm1 = nn.GroupNorm(32, out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1)
        self.norm2 = nn.GroupNorm(32, out_ch)
        # 채널 수가 다를 경우 skip connection을 위한 1x1 conv
        self.skip = nn.Conv2d(in_ch, out_ch, 1) if in_ch != out_ch else nn.Identity()

    def forward(self, x):
        h = F.silu(self.norm1(self.conv1(x)))
        h = F.silu(self.norm2(self.conv2(h)))
        return h + self.skip(x)


class Encoder(nn.Module):
    """
    이미지 → 잠재 공간 인코더
    입력:  [B, 3, 256, 256]
    출력:  mean [B, 4, 32, 32], logvar [B, 4, 32, 32]
    다운샘플링 팩터 f=8 (256 → 128 → 64 → 32)
    """
    def __init__(self, in_ch=3, latent_ch=4, base_ch=128):
        super().__init__()
        self.encoder = nn.Sequential(
            # 256×256 → 128×128
            nn.Conv2d(in_ch, base_ch, 3, stride=2, padding=1),
            ResBlock(base_ch, base_ch),
            # 128×128 → 64×64
            nn.Conv2d(base_ch, base_ch * 2, 3, stride=2, padding=1),
            ResBlock(base_ch * 2, base_ch * 2),
            # 64×64 → 32×32
            nn.Conv2d(base_ch * 2, base_ch * 4, 3, stride=2, padding=1),
            ResBlock(base_ch * 4, base_ch * 4),
        )
        # mean과 logvar를 동시에 출력 (채널 2배)
        self.to_moments = nn.Conv2d(base_ch * 4, latent_ch * 2, 1)

    def forward(self, x):
        h = self.encoder(x)
        moments = self.to_moments(h)
        # 채널 차원에서 mean, logvar로 분리
        mean, logvar = moments.chunk(2, dim=1)  # 각각 [B, 4, 32, 32]
        return mean, logvar


class Decoder(nn.Module):
    """
    잠재 공간 → 이미지 디코더
    입력:  [B, 4, 32, 32]
    출력:  [B, 3, 256, 256]
    업샘플링 팩터 f=8 (32 → 64 → 128 → 256)
    """
    def __init__(self, latent_ch=4, out_ch=3, base_ch=128):
        super().__init__()
```

```python
        self.from_latent = nn.Conv2d(latent_ch, base_ch * 4, 1)
        self.decoder = nn.Sequential(
            ResBlock(base_ch * 4, base_ch * 4),
            # 32×32 → 64×64
            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(base_ch * 4, base_ch * 2, 3, padding=1),
            ResBlock(base_ch * 2, base_ch * 2),
            # 64×64 → 128×128
            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(base_ch * 2, base_ch, 3, padding=1),
            ResBlock(base_ch, base_ch),
            # 128×128 → 256×256
            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(base_ch, base_ch, 3, padding=1),
            ResBlock(base_ch, base_ch),
        )
        self.to_image = nn.Conv2d(base_ch, out_ch, 1)

    def forward(self, z):
        h = self.from_latent(z)
        h = self.decoder(h)
        return self.to_image(h)


class KLVAE(nn.Module):
    """
    간소화된 KL-VAE (Stable Diffusion의 오토인코더)

    주요 구성:
    - Encoder: 이미지 → (mean, logvar)
    - Reparameterization trick: z = mean + std * epsilon
    - Decoder: z → 복원 이미지
    - Scaling factor: 잠재 공간의 표준편차 정규화
    """
    def __init__(self, scaling_factor=0.18215):
        super().__init__()
        self.encoder = Encoder()
        self.decoder = Decoder()
        self.scaling_factor = scaling_factor  # SD 1.x 기본값

    def encode(self, x):
        """이미지를 잠재 공간으로 인코딩 (스케일링 포함)"""
        mean, logvar = self.encoder(x)
        # Reparameterization trick: 미분 가능한 샘플링
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        z = mean + std * eps
        # Scaling factor 적용
        z = z * self.scaling_factor
        return z, mean, logvar

    def decode(self, z):
        """잠재 표현을 이미지로 디코딩 (역스케일링 포함)"""
        z = z / self.scaling_factor
        return self.decoder(z)

    def forward(self, x):
        z, mean, logvar = self.encode(x)
        x_recon = self.decode(z)
        return x_recon, mean, logvar

    @staticmethod
```

```
    def kl_loss(mean, logvar):
        """KL Divergence: q(z|x) || N(0,I)"""
        return -0.5 * torch.mean(1 + logvar - mean.pow(2) - logvar.exp())


# === 사용 예시 ===
if __name__ == "__main__":
    vae = KLVAE()
    # 가상의 배치: [B=2, C=3, H=256, W=256]
    x = torch.randn(2, 3, 256, 256)

    # 인코딩
    z, mean, logvar = vae.encode(x)
    print(f"입력 이미지 크기: {x.shape}")        # [2, 3, 256, 256]
    print(f"잠재 표현 크기:   {z.shape}")         # [2, 4, 32, 32]
    print(f"차원 축소 비율:   {x.numel() / z.numel():.0f}배")  # 48배

    # 디코딩
    x_recon = vae.decode(z)
    print(f"복원 이미지 크기: {x_recon.shape}")   # [2, 3, 256, 256]

    # Loss 계산
    recon_loss = F.mse_loss(x_recon, x)
    kl = KLVAE.kl_loss(mean, logvar)
    total_loss = recon_loss + 1e-6 * kl  # 약한 KL 페널티
    print(f"복원 Loss: {recon_loss.item():.4f}, KL Loss: {kl.item():.4f}")
```

---

3. 잠재 공간 디퓨전 (Stage 2)

> 💡 **왜 Stage 1과 Stage 2를 완전히 분리하는가?** 이 2-stage 설계는 단순히 편의를 위한 것이 아니라 **근본적인 이점**이 있다. 오토인코더(Stage 1)의 학습 목표는 "이미지를 잘 압축/복원하는 것"이고, 디퓨전(Stage 2)의 학습 목표는 "잠재 공간의 분포를 모델링하는 것"이다. 이 두 목표는 서로 독립적이므로, 분리하면 각각이 더 효율적으로 학습된다. 더 중요한 점은, Stage 1이 완료되면 잠재 공간이 **고정된 좌표계**가 되므로, Stage 2의 디퓨전 모델은 "안정적인 공간에서의 분포 학습"이라는 well-defined된 문제를 풀게 된다. 만약 두 단계를 동시에 학습하면, 잠재 공간의 의미가 매 iteration마다 변하여 디퓨전 학습이 수렴하기 극히 어려워진다.

3.1 학습

오토인코더를 고정한 뒤, 잠재 공간에서 DDPM을 학습:

```
1. x₀ → [Encoder E] → z₀ (잠재 표현)
2. z₀ ← z₀ × scaling_factor (정규화)
3. t ~ Uniform{1,...,T}, ε ~ N(0,I)
4. z_t = √ᾱ_t · z₀ + √(1-ᾱ_t) · ε
5. 텍스트 → [CLIP Text Encoder] → c
6. ε̂= ε_θ(z_t, t, c)          # 조건부 U-Net/DiT
7. Loss = ‖ε - ε̂‖²
```

DDPM과 **동일한 수식**, 다만 x 대신 z에서 수행:

$$\mathcal{L} = \mathbb{E}_{t,z_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(z_t, t, c)\|^2\right]$$

위 수식의 각 요소를 정확히 이해하자:

- $t \sim \text{Uniform}\{1, \ldots, T\}$: 타임스텝을 균일하게 샘플링한다. 모든 노이즈 수준에서 고르게 학습하기 위해서이다.

- $z_0 = s \cdot E(x_0)$: 원본 이미지를 인코딩하고 scaling factor s를 곱한 잠재 표현이다.
- $\epsilon \sim \mathcal{N}(0, I)$: 표준 가우시안에서 샘플링한 노이즈이다. 이것이 U-Net이 예측해야 할 **타겟**이다.
- $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$: forward process로, z_0에 스케줄에 따라 노이즈를 추가한 결과이다. $\bar{\alpha}_t$가 작을수록(= t가 클수록) 노이즈가 더 많이 섞인다.
- $\epsilon_\theta(z_t, t, c)$: U-Net이 예측하는 노이즈이다. 입력으로 노이즈가 섞인 잠재 표현 z_t, 현재 타임스텝 t, 텍스트 조건 c를 받는다.

이 Loss가 작동하는 직관: U-Net이 "z_t에서 z_0로 가려면 어떤 방향으로 얼마나 가야 하는가?"를 학습한다고 볼 수 있다. ϵ은 z_0에서 z_t를 만들기 위해 추가된 노이즈이므로, ϵ을 정확히 예측하면 z_t에서 z_0를 복원할 수 있다: $z_0 = (z_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)/\sqrt{\bar{\alpha}_t}$.

조건부 생성의 핵심 — 텍스트 조건 c의 역할:

조건 c가 Loss 수식에 **직접** 등장하지 않는다는 점에 주목하자. c는 오직 U-Net의 입력으로만 들어간다. 그렇다면 U-Net은 왜 c에 주의를 기울이는가? 그 이유는, **같은 z_t라도 다른 c가 주어지면 다른 ϵ이 정답**이기 때문이다. 예를 들어, 노이즈가 많이 섞인 z_t에서 "고양이" 조건이 주어지면 고양이를 향해 디노이징해야 하고, "강아지" 조건이 주어지면 강아지를 향해 디노이징해야 한다. 학습 데이터에서 각 이미지가 텍스트와 쌍을 이루므로, U-Net은 자연스럽게 c를 참고하여 노이즈를 예측하게 된다.

3.2 생성

```
1. 텍스트 → [CLIP] → c
2. z_T ~ N(0, I)                          # 잠재 공간의 노이즈
3. for t = T, ..., 1:
     ε̃= CFG(ε_θ, z_t, t, c, w)          # Classifier-Free Guidance
     z_{t-1} = denoise_step(z_t, ε̃, t)
4. z₀ ← z₀ / scaling_factor               # 역정규화
5. x₀ = Decoder(z₀)                       # 잠재 → 이미지
```

생성 과정에서 CFG(Classifier-Free Guidance)가 핵심적인 역할을 한다. CFG의 수식을 다시 살펴보자:

$$\tilde{\epsilon} = \epsilon_\theta(z_t, t, \varnothing) + w \cdot (\epsilon_\theta(z_t, t, c) - \epsilon_\theta(z_t, t, \varnothing))$$

여기서 $\varnothing$는 빈 조건(null condition, 빈 텍스트)이고, w는 guidance scale이다. 이 수식의 의미를 직관적으로 이해하면:

- $\epsilon_\theta(z_t, t, \varnothing)$: "아무 조건 없이 자연스러운 이미지를 만들어라" (무조건부 방향)
- $\epsilon_\theta(z_t, t, c) - \epsilon_\theta(z_t, t, \varnothing)$: "텍스트 조건이 추가되면 얼마나 달라지는가?" (조건의 효과)
- w를 곱해서 조건의 효과를 **증폭**: $w = 7.5$면 조건 방향으로 7.5배 강하게 이동

이것은 마치 "고양이를 그려라"라는 지시를 받았을 때, $w = 1$이면 "적당히 고양이같은" 이미지를, $w = 7.5$면 "확실히 고양이인" 이미지를 생성하는 것과 같다. 대가는 **다양성 감소** — w가 높을수록 모든 생성 결과가 비슷해진다.

> 💡 **CFG의 실용적 의미: 왜 매 스텝마다 U-Net을 2번 호출하는가?** CFG를 사용하면 매 디노이징 스텝에서 U-Net을 **조건부 1회 + 무조건부 1회 = 총 2회** 호출해야 한다. 이는 생성 시간을 2배로 만든다. 실제 구현에서는 조건부와 무조건부 입력을 **배치로 합쳐서** 1회의 forward pass로 처리하는 최적화를 사용한다 (batch size를 2배로 만들어서 한 번에 처리). 이것이 SD 생성 시 메모리가 학습 시보다 더 필요한 이유 중 하나이다.

3.3 U-Net 구조 (SD 1.x/2.x)

```
z_t (32×32×4) + t + c
        ↓
┌──────────────────────────────────────┐
│  Encoder                  Decoder    │
│  ┌──────────┐            ┌──────────┐ │
│  │32×32×320│── skip ──→│32×32×320│ │
│  │ResBlock  │            │ResBlock  │ │
│  │CrossAttn │            │CrossAttn │ │
│  │↓ Down    │            │↑ Up      │ │
│  ├──────────┤            ├──────────┤ │
│  │16×16×640│── skip ──→│16×16×640│ │
│  │ResBlock  │            │ResBlock  │ │
│  │CrossAttn │            │CrossAttn │ │
│  │↓ Down    │            │↑ Up      │ │
│  ├──────────┤            ├──────────┤ │
│  │8×8×1280 │── skip ──→│8×8×1280 │ │
│  │ResBlock  │            │ResBlock  │ │
│  │CrossAttn │            │CrossAttn │ │
│  │↓ Down    │            │↑ Up      │ │
│  └──────────┘            └──────────┘ │
│       └──→[Bottleneck 8×8]──┘         │
└──────────────────────────────────────┘
        ↓
   ε^(32×32×4)
```

- **Cross-Attention**: 각 레벨에서 텍스트 조건 c 주입
- **Self-Attention**: 이미지 내부 관계 모델링
- 파라미터: ~860M (SD 1.x), ~865M (SD 2.x)

Cross-Attention의 동작 원리 상세 설명:

Cross-Attention은 텍스트 정보가 이미지 생성에 영향을 미치는 **핵심 메커니즘**이다. 구체적인 동작을 단계별로 살펴보자.

U-Net의 특정 레이어에서 공간 특징 맵(spatial feature map)이 $\phi \in \mathbb{R}^{(h \times w) \times d_{\text{model}}}$이고, CLIP 텍스트 임베딩이 $c \in \mathbb{R}^{L \times d_{\text{text}}}$일 때:

$$Q = \phi W_Q, \quad K = \tau_\theta(c) W_K, \quad V = \tau_\theta(c) W_V$$

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

여기서:

- Q (Query): **이미지의 각 공간 위치**에서 "나는 어떤 텍스트 정보가 필요한가?"를 나타낸다
- K (Key): **텍스트의 각 토큰**에서 "나는 이런 종류의 정보를 제공한다"를 나타낸다
- V (Value): **텍스트의 각 토큰**의 실제 정보 내용이다
- τ_θ: 텍스트 임베딩을 U-Net 특징 공간으로 변환하는 **투영 레이어(projection layer)**이다

> 💡 **왜 τ_θ 투영이 필요한가?** CLIP ViT-L/14의 출력 차원은 768이고, U-Net의 내부 특징 차원은 레벨에 따라 320/640/1280이다. 이 차원이 다르므로 직접 Attention을 계산할 수 없다. τ_θ는 단순한 Linear 레이어로, CLIP 임베딩을 U-Net 내부 차원에 맞게 변환한다. 단, 이 투영은 단순한 차원 변환 이상의 역할을 한다 — CLIP은 "텍스트-이미지 유사도"에 최적화된 표현을 학습했지만, U-Net은 "디노이징에 필요한 공간적 정보"가 필요하므로, τ_θ는 이 **의미적 갭을 연결**하는 역할을 한다.

직관적 예시: "a red cat sitting on a blue chair"라는 프롬프트가 주어졌을 때:

- 이미지의 고양이 위치에 있는 Query는 "red"와 "cat" 토큰의 Key와 높은 유사도를 가진다
- 의자 위치에 있는 Query는 "blue"와 "chair" 토큰과 높은 유사도를 가진다

- softmax 후 가중치에 따라, 각 공간 위치에 관련된 텍스트 정보가 주입된다

이 메커니즘 덕분에 텍스트의 각 단어가 이미지의 **올바른 공간 위치에** 영향을 미칠 수 있다. Self-Attention만으로는 불가능한 **공간적으로 정밀한 조건부 생성**이 가능해진다.

> 💡 **왜 CLIP 텍스트 인코더를 사용하는가?** CLIP은 4억 개의 이미지-텍스트 쌍으로 대조 학습(contrastive learning)되었다. 따라서 CLIP의 텍스트 표현은 단순한 언어 모델의 표현과 달리, **시각적 개념과 이미 정렬**되어 있다. "golden retriever"라는 텍스트의 CLIP 임베딩은 실제 골든 리트리버 이미지의 CLIP 임베딩과 가깝다. 이 사전 정렬 덕분에 디퓨전 모델은 텍스트-이미지 매핑을 **처음부터 배울 필요 없이**, CLIP이 이미 학습한 텍스트-이미지 관계를 활용할 수 있다.

타임스텝 조건 주입 — Sinusoidal Embedding + AdaGN:

타임스텝 t는 현재 노이즈 수준을 나타내며, U-Net이 "지금 얼마나 노이즈가 많은 상태인지"를 알아야 올바른 디노이징을 수행할 수 있다. t는 Transformer의 positional encoding과 동일한 sinusoidal embedding으로 인코딩된 뒤, MLP를 거쳐 각 ResBlock의 **Adaptive Group Normalization (AdaGN)**에 주입된다:

$$\text{AdaGN}(h, t) = t_s \cdot \text{GroupNorm}(h) + t_b$$

여기서 t_s와 t_b는 타임스텝 임베딩에서 예측된 scale과 bias이다. 이를 통해 U-Net의 각 레이어가 현재 노이즈 수준에 맞춘 처리를 수행한다.

3.4 PyTorch 구현: Latent Diffusion 학습 루프

⚠️ **참고**: 아래 코드는 Latent Diffusion 학습의 핵심 흐름을 보여주기 위한 간소화 버전이다. 실제 구현에서는 EMA, gradient accumulation, mixed precision 등이 추가된다.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader


class LatentDiffusionTrainer:
    """
    Latent Diffusion Model 학습기

    핵심 흐름:
    1. 이미지를 VAE 인코더로 잠재 공간에 매핑
    2. 잠재 표현에 노이즈를 추가 (forward process)
    3. U-Net으로 노이즈 예측
    4. 예측된 노이즈와 실제 노이즈의 MSE Loss
    """
    def __init__(
        self,
        vae,            # 사전학습된 KL-VAE (고정됨)
        unet,           # 학습할 노이즈 예측 네트워크
        text_encoder,   # 사전학습된 CLIP 텍스트 인코더 (고정됨)
        num_timesteps=1000,
        beta_start=0.00085,
        beta_end=0.012,
        device='cuda'
    ):
        self.device = device
        self.num_timesteps = num_timesteps

        # === 모델 설정 ===
        # VAE와 Text Encoder는 고정 (학습하지 않음)
        self.vae = vae.to(device).eval()
        for p in self.vae.parameters():
            p.requires_grad_(False)

        self.text_encoder = text_encoder.to(device).eval()
        for p in self.text_encoder.parameters():
            p.requires_grad_(False)

        # U-Net만 학습
        self.unet = unet.to(device).train()

        # === 노이즈 스케줄 (Scaled Linear Schedule) ===
        # SD 1.x에서 사용하는 스케줄: beta가 sqrt 공간에서 선형
        betas = torch.linspace(beta_start ** 0.5, beta_end ** 0.5, num_timesteps) **
        alphas = 1.0 - betas
        # 누적 곱: ᾱ_t = α₁ × α₂ × ... × α_t
        self.alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)

        # scaling factor (SD 1.x 기본값)
        self.scaling_factor = 0.18215

    @torch.no_grad()
    def encode_images(self, images):
        """이미지 배치를 잠재 공간으로 인코딩"""
        # VAE 인코더를 통해 mean, logvar 추출
        z, mean, logvar = self.vae.encode(images)
        return z  # [B, 4, 32, 32]

    @torch.no_grad()
    def encode_text(self, text_tokens):
        """텍스트 토큰을 CLIP 임베딩으로 변환"""
```

```python
        return self.text_encoder(text_tokens)  # [B, seq_len, dim]

    def training_step(self, images, text_tokens):
        """
        단일 학습 스텝

        Args:
            images: [B, 3, 256, 256] 원본 이미지
            text_tokens: [B, seq_len] 토크나이즈된 텍스트

        Returns:
            loss: 스칼라 학습 손실값
        """
        batch_size = images.shape[0]

        # 1단계: 이미지를 잠재 공간으로 인코딩
        z_0 = self.encode_images(images)  # [B, 4, 32, 32]

        # 2단계: 텍스트 조건 인코딩
        c = self.encode_text(text_tokens)  # [B, seq_len, 768]

        # 3단계: 랜덤 타임스텝 샘플링
        t = torch.randint(0, self.num_timesteps, (batch_size,), device=self.device)

        # 4단계: 노이즈 샘플링 및 forward process
        noise = torch.randn_like(z_0)
        # z_t = √ᾱ_t · z₀ + √(1 - ᾱ_t) · ε
        alpha_t = self.alphas_cumprod[t][:, None, None, None]  # [B, 1, 1, 1]
        z_t = torch.sqrt(alpha_t) * z_0 + torch.sqrt(1 - alpha_t) * noise

        # 5단계: 노이즈 예측 (U-Net)
        noise_pred = self.unet(z_t, t, c)  # ε̂= ε_θ(z_t, t, c)

        # 6단계: 손실 계산 (Simple Loss)
        loss = F.mse_loss(noise_pred, noise)

        return loss

    def train_epoch(self, dataloader, optimizer):
        """한 에포크 학습"""
        total_loss = 0
        for batch_idx, (images, text_tokens) in enumerate(dataloader):
            images = images.to(self.device)
            text_tokens = text_tokens.to(self.device)

            optimizer.zero_grad()
            loss = self.training_step(images, text_tokens)
            loss.backward()
            # Gradient clipping (학습 안정성)
            torch.nn.utils.clip_grad_norm_(self.unet.parameters(), max_norm=1.0)
            optimizer.step()

            total_loss += loss.item()

            if batch_idx % 100 == 0:
                print(f"  배치 {batch_idx}: loss = {loss.item():.6f}")

        return total_loss / len(dataloader)


# === 학습 실행 예시 ===
if __name__ == "__main__":
```

```
# 모델 초기화 (실제로는 사전학습된 가중치 로드)
# vae = load_pretrained_vae("stabilityai/sd-vae-ft-mse")
# text_encoder = load_pretrained_clip("openai/clip-vit-large-patch14")
# unet = UNet2DConditionModel(...)

# trainer = LatentDiffusionTrainer(vae, unet, text_encoder)
# optimizer = torch.optim.AdamW(unet.parameters(), lr=1e-4, weight_decay=0.01)

# for epoch in range(100):
#     avg_loss = trainer.train_epoch(dataloader, optimizer)
#     print(f"에포크 {epoch}: 평균 Loss = {avg_loss:.6f}")

print("학습 루프 구조 확인 완료 (실제 학습에는 대규모 데이터셋 필요)")
```

---

4. 전체 파이프라인 정리

> 💡 **왜 파이프라인 전체를 한눈에 이해해야 하는가?** LDM은 여러 독립적인 구성 요소가 연결된 시스템이다. 각 구성 요소를 개별적으로 이해하는 것과 **전체 흐름에서 각 구성 요소의 역할**을 이해하는 것은 다르다. 디버깅, 성능 최적화, 새로운 기능 추가(ControlNet, IP-Adapter 등) 시에는 반드시 전체 파이프라인 수준에서 이해해야 한다. 예를 들어, 생성 이미지가 흐릿하면 "VAE 디코더 문제인가? 디퓨전 스텝 부족인가? CFG가 너무 낮은가?"를 판단하려면 전체 흐름을 알아야 한다.

4.1 학습 단계

```
Phase 1: 오토인코더 학습 (1회)
──────────────────────────────
이미지 → [Encoder] → z → [Decoder] → 이미지 복원
Loss = MSE + KL + LPIPS + GAN
→ 학습 완료 후 Encoder/Decoder 고정

Phase 2: 디퓨전 학습
────────────────────
이미지 → [고정 Encoder] → z₀
텍스트 → [고정 CLIP]    → c
z_t = √ᾱ_t z₀ + √(1-ᾱ_t) ε
Loss = ‖ε - ε_θ(z_t, t, c)‖²
→ U-Net/DiT만 학습
```

두 Phase의 학습 규모 비교:

	Phase 1 (오토인코더)	**Phase 2 (디퓨전)**
학습 대상	Encoder + Decoder (~80M params)	U-Net (~860M params)
학습 데이터	이미지만 필요	이미지 + 텍스트 쌍 필요
학습 기간	~수일 (1회만)	~수주 (핵심 학습)
재사용성	여러 디퓨전 모델에 공유 가능	모델별 고유

Phase 1은 한 번만 학습하면 되고, 실제로 SD 1.x와 2.x는 **동일한 VAE**를 사용했다. 이는 2-stage 설계의 실용적 장점이다 — VAE를 개선하면 기존 디퓨전 모델의 출력 품질도 함께 향상된다 (실제로 Stability AI는 SD 1.x용 VAE를 이후에 재학습한 `sd-vae-ft-mse`를 별도로 공개했다).

4.2 생성 단계

```
텍스트 "a cat on a throne"
    ↓
[CLIP Text Encoder]
    ↓
c (조건 임베딩)
    ↓                          ↓
z_T ~ N(0,I)  ←── [CFG + DDPM/DDIM Sampling, 50 steps] ──→ z₀
                                                              ↓
                                                      [VAE Decoder]
                                                              ↓
                                                      이미지 256×256
```

4.3 PyTorch 구현: CFG를 포함한 전체 생성 파이프라인

```python
import torch
import torch.nn.functional as F
from PIL import Image
import numpy as np


class LatentDiffusionSampler:
    """
    Latent Diffusion Model 생성(샘플링) 파이프라인

    DDIM 샘플링 + Classifier-Free Guidance를 사용한
    텍스트 → 이미지 생성의 전체 과정을 구현
    """
    def __init__(self, vae, unet, text_encoder, device='cuda'):
        self.vae = vae.to(device).eval()
        self.unet = unet.to(device).eval()
        self.text_encoder = text_encoder.to(device).eval()
        self.device = device

        # 노이즈 스케줄 설정 (SD 1.x와 동일)
        self.num_timesteps = 1000
        betas = torch.linspace(0.00085 ** 0.5, 0.012 ** 0.5, 1000) ** 2
        alphas = 1.0 - betas
        self.alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
        self.scaling_factor = 0.18215

    @torch.no_grad()
    def generate(
        self,
        prompt_tokens,              # 텍스트 프롬프트 토큰 [1, seq_len]
        num_inference_steps=50,     # DDIM 샘플링 스텝 수
        guidance_scale=7.5,         # CFG 스케일 (w)
        latent_height=32,           # 잠재 공간 높이 (= 이미지 높이 / 8)
        latent_width=32,            # 잠재 공간 너비 (= 이미지 너비 / 8)
        seed=42
    ):
        """
        텍스트 조건부 이미지 생성

        핵심 과정:
        1. 텍스트를 CLIP으로 인코딩
        2. 순수 노이즈에서 시작
        3. CFG를 적용한 DDIM 역과정으로 디노이징
        4. VAE 디코더로 잠재 표현을 이미지로 변환
        """
        # 재현성을 위한 시드 설정
        generator = torch.Generator(device=self.device).manual_seed(seed)

        # === 1단계: 텍스트 조건 인코딩 ===
        # 조건부 임베딩 (실제 프롬프트)
        c = self.text_encoder(prompt_tokens.to(self.device))  # [1, seq_len, 768]

        # 비조건부 임베딩 (빈 프롬프트 - CFG에 필요)
        # 실제로는 빈 문자열("")을 토크나이즈한 것
        uncond_tokens = torch.zeros_like(prompt_tokens)  # 패딩 토큰
        uc = self.text_encoder(uncond_tokens.to(self.device))

        # === 2단계: 초기 잠재 노이즈 생성 ===
        z_t = torch.randn(
            1, 4, latent_height, latent_width,
            device=self.device, generator=generator
        )
```

```python
        # === 3단계: DDIM 타임스텝 선택 ===
        # 1000 스텝에서 균등하게 num_inference_steps개 선택
        step_ratio = self.num_timesteps // num_inference_steps
        timesteps = torch.arange(num_inference_steps, device=self.device) * step_rati
        timesteps = timesteps.flip(0)  # 역순: T → 0

        # === 4단계: 디노이징 루프 (DDIM + CFG) ===
        for i, t in enumerate(timesteps):
            t_batch = t.unsqueeze(0)  # [1]

            # --- Classifier-Free Guidance ---
            # 비조건부 예측: ε_θ(z_t, t, ∅)
            noise_uncond = self.unet(z_t, t_batch, uc)

            # 조건부 예측: ε_θ(z_t, t, c)
            noise_cond = self.unet(z_t, t_batch, c)

            # CFG 결합: ε˜= ε_uncond + w * (ε_cond - ε_uncond)
            noise_pred = noise_uncond + guidance_scale * (noise_cond - noise_uncond)

            # --- DDIM 업데이트 스텝 ---
            alpha_t = self.alphas_cumprod[t.long()]
            # 이전 타임스텝의 alpha (마지막 스텝이면 alpha=1)
            if i + 1 < len(timesteps):
                alpha_prev = self.alphas_cumprod[timesteps[i + 1].long()]
            else:
                alpha_prev = torch.tensor(1.0, device=self.device)

            # x₀ 예측: z₀_pred = (z_t - √(1-ᾱ_t) * ε) / √ᾱ_t
            z_0_pred = (z_t - torch.sqrt(1 - alpha_t) * noise_pred) / torch.sqrt(alpl

            # DDIM 업데이트 (결정론적, η=0)
            # z_{t-1} = √ᾱ_{t-1} * z₀_pred + √(1-ᾱ_{t-1}) * ε˜
            z_t = (torch.sqrt(alpha_prev) * z_0_pred
                   + torch.sqrt(1 - alpha_prev) * noise_pred)

            if i % 10 == 0:
                print(f"  디노이징 스텝 {i}/{num_inference_steps}, t={t.item()}")

        # === 5단계: 잠재 표현 → 이미지 디코딩 ===
        image = self.vae.decode(z_t)  # [1, 3, 256, 256]

        # [-1, 1] → [0, 255] 범위로 변환
        image = (image.clamp(-1, 1) + 1) / 2 * 255
        image = image[0].permute(1, 2, 0).cpu().numpy().astype(np.uint8)

        return Image.fromarray(image)


# === 사용 예시 ===
if __name__ == "__main__":
    # 실제 사용 시:
    # sampler = LatentDiffusionSampler(vae, unet, text_encoder)
    # image = sampler.generate(
    #     prompt_tokens=tokenizer("a cat sitting on a throne, oil painting"),
    #     guidance_scale=7.5,
    #     num_inference_steps=50
    # )
    # image.save("generated_cat.png")

    print("=== 생성 파이프라인 구조 ===")
```

```
print("1. 텍스트 → CLIP 인코딩")
print("2. z_T ~ N(0, I) 초기 노이즈")
print("3. DDIM 역과정 (50 스텝) + CFG (w=7.5)")
print("4. VAE 디코딩 → 최종 이미지")
```

⚠️ **실전 팁**: `guidance_scale` 값에 따른 효과:

- **w = 1.0**: CFG 없음 (조건 신호 약함, 다양성 높음)
- **w = 7.5**: SD 1.x 기본값 (품질과 다양성의 균형)
- **w = 15+**: 프롬프트에 과도하게 충실 (색이 과포화되거나 아티팩트 발생 가능)

---

5. Stable Diffusion 버전별 진화

> 💡 **왜 버전별 진화를 이해해야 하는가?** SD의 각 버전은 이전 버전의 **구체적인 한계**를 해결하기 위해 특정 구성 요소를 교체하거나 개선했다. 이 진화 과정을 이해하면, 각 기술적 선택이 "왜" 이루어졌는지를 알 수 있고, 미래의 모델이 어떤 방향으로 발전할지 예측할 수 있다. 단순히 "SD 3는 DiT를 사용한다"를 아는 것과 "왜 U-Net에서 DiT로 바꿨는지"를 이해하는 것은 근본적으로 다른 수준의 지식이다.

5.1 SD 1.x (2022.08)

📄 Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

항목	상세
Backbone	U-Net (~860M)
텍스트 인코더	CLIP ViT-L/14 (768차원)
예측 대상	ϵ-prediction
해상도	512×512
학습 데이터	LAION-5B 부분집합
샘플러	DDPM, DDIM, Euler, DPM-Solver

SD 1.x의 알려진 한계점:

1. **텍스트 이해도 제한**: CLIP ViT-L/14의 토큰 길이가 77로 제한되고, 복잡한 공간 관계("왼쪽에 A, 오른쪽에 B")를 잘 이해하지 못했다. 이는 CLIP이 이미지-텍스트 **전체 유사도** 최적화에 맞춰져 있어, 텍스트의 **세부 구성 요소**를 구분하는 능력이 부족하기 때문이다.
2. **해상도 고정**: 512×512에서만 학습되어, 다른 해상도에서는 반복 패턴이나 기형적 구조가 발생했다.
3. **ϵ-prediction의 한계**: 극단적 노이즈 수준(t가 매우 크거나 작을 때)에서 gradient가 불안정했다.

5.2 SD 2.x (2022.11)

📄 Stability AI, "Stable Diffusion 2.0 Release"

변경	상세
텍스트 인코더	OpenCLIP ViT-H/14 (1024차원)
예측 대상	v-prediction (노이즈와 원본의 가중 결합)
해상도	512 또는 768

> 💡 **왜 OpenCLIP으로 바꿨는가?** SD 1.x의 CLIP ViT-L/14는 OpenAI가 학습한 것으로, 학습 데이터에 NSFW 필터링과 특정 편향이 있었다. OpenCLIP은 오픈소스 프로젝트로, 더 다양하고 큰 데이터셋으로 학습되어 표현 능력이 향상되었다. 또한 ViT-H/14는 ViT-L/14보다 모델 크기가 크고(632M vs 428M), 임

> 베딩 차원도 1024로 늘어나 더 풍부한 텍스트 표현이 가능했다. 그러나 실제로는 커뮤니티에서 SD 2.x의 프롬프트 반응이 SD 1.x보다 "다르다"는 평가가 많았는데, 이는 OpenCLIP의 학습 데이터 분포가 달랐기 때문이다.

v-prediction 상세 설명:

v-prediction:

$$v_t = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,x_0$$

ϵ-prediction과 수학적으로 동치이나, 수치적으로 더 안정적일 수 있다.

> 💡 **왜 v-prediction이 ϵ-prediction보다 수치적으로 안정적인가?** 이를 이해하려면 각 예측 대상의 **gradient 크기**가 타임스텝 t에 따라 어떻게 변하는지를 분석해야 한다.
>
> ϵ-prediction에서, Loss의 gradient는 다음에 비례한다:
>
> $$\frac{\partial \mathcal{L}_\epsilon}{\partial \theta} \propto \frac{1}{\sqrt{1-\bar{\alpha}_t}}(\epsilon - \epsilon_\theta)$$
>
> t가 매우 작을 때(= 노이즈가 거의 없을 때), $\bar{\alpha}_t \approx 1$이므로 $\sqrt{1-\bar{\alpha}_t} \approx 0$이 되어 gradient가 **폭발**한다. 반대로 t가 매우 클 때(= 거의 순수 노이즈), 노이즈 예측이 쉬워서 gradient가 매우 작다.
>
> v-prediction에서 v_t는 ϵ과 x_0의 **가중 결합**이다:
>
> - t가 작을 때: $\sqrt{\bar{\alpha}_t} \approx 1$, $\sqrt{1-\bar{\alpha}_t} \approx 0 \to v_t \approx \epsilon$ (노이즈에 가까움)
> - t가 클 때: $\sqrt{\bar{\alpha}_t} \approx 0$, $\sqrt{1-\bar{\alpha}_t} \approx 1 \to v_t \approx -x_0$ (원본에 가까움)
>
> 이 "적응적 보간" 덕분에 gradient 크기가 모든 t에서 **더 균일**해진다. 극단적인 t 값에서의 gradient 폭발/소멸 문제가 완화되어, 학습이 더 안정적이다.

5.3 SDXL (2023.07)

📄 Podell et al., "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis", 2023

변경	상세
U-Net	2.6B (3배 확대)
텍스트 인코더	CLIP ViT-L + OpenCLIP ViT-bigG **듀얼**
해상도	1024×1024 기본
추가	Refiner 모델 (2단계 생성), 해상도/크롭 조건

> 💡 **왜 듀얼 텍스트 인코더를 사용하는가?** 단일 텍스트 인코더는 텍스트의 모든 측면(의미, 스타일, 공간 관계, 세부 속성)을 하나의 임베딩에 담아야 한다. 듀얼 인코더는 **서로 다른 관점의 텍스트 표현**을 제공한다: CLIP ViT-L은 SD 1.x 시절부터 사용되어 **일반적인 시각적 개념**에 강하고, OpenCLIP ViT-bigG는 2.5B 파라미터의 대형 모델로 **세밀한 의미적 뉘앙스**를 포착한다. 두 표현을 concat하면 Cross-Attention에서 U-Net이 두 관점 모두를 참조할 수 있다.

텍스트 조건이 두 인코더의 출력을 **concat**하여 사용:

$$c = [c_{\text{CLIP-L}};\ c_{\text{OpenCLIP-bigG}}]$$

구체적으로, CLIP ViT-L은 768차원, OpenCLIP ViT-bigG는 1280차원의 임베딩을 출력하므로, concat된 조건 벡터는 각 토큰당 $768 + 1280 = 2048$차원이 된다. Cross-Attention의 Key와 Value는 이 2048차원 벡터에서 투영된다.

SDXL의 추가 혁신 — 해상도 조건과 크롭 조건:

SDXL은 학습 데이터의 **원래 해상도**와 **크롭 좌표**를 추가 조건으로 U-Net에 주입한다. 이전 모델들은 모든 학습 이미지를 동일 크기로 리사이즈/크롭했는데, 이로 인해 모델이 "모든 이미지가 정사각형이고 중앙 정렬"이라고 학습하게 되었다. 해상도/크롭 조건을 추가하면, 모델은 "이 이미지는 원래 1920×1080이었고, 중앙을 크롭한 것"이라는 메타데이터를 알게 되어, 생성 시 원하는 종횡비와 구도를 더 정밀하게 제어할 수 있다.

Refiner 모델:

SDXL은 Base 모델로 전체 구조를 생성한 뒤, Refiner 모델로 **고주파 디테일을 정제**하는 2단계 전략을 사용한다. Refiner는 Base가 생성한 잠재 표현에 약간의 노이즈를 다시 추가한 뒤(img2img 방식), 디테일에 특화된 별도 U-Net으로 디노이징한다. 이는 하나의 모델이 "전체 구조"와 "미세 디테일" 모두를 완벽하게 처리하기 어렵다는 관찰에 기반한다.

5.4 SD 3 (2024.02) — 패러다임 전환

📄 Esser et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", ICML 2024

변경	상세
Backbone	**MM-DiT** (U-Net → Transformer)
텍스트 인코더	CLIP ViT-L + CLIP ViT-bigG + **T5-XXL** 트리플
학습 방식	**Flow Matching** (DDPM → ODE)
구조	텍스트와 이미지 토큰을 동등하게 처리

> **SD 3는 세 가지 패러다임 전환이 동시에 일어남**: U-Net→DiT, DDPM→Flow Matching, 단일 인코더→트리플 인코더.

패러다임 전환 1: U-Net → MM-DiT (Multi-Modal Diffusion Transformer)

> 💡 **왜 U-Net에서 Transformer로 바꿨는가?** U-Net은 원래 의료 영상 세그먼테이션을 위해 설계된 아키텍처이다. 디퓨전에 성공적으로 적용되었지만, 근본적인 한계가 있다:
>
> 1. **확장성(Scalability)**: U-Net은 채널 수를 늘리면 파라미터가 빠르게 증가하지만, 성능 향상은 선형적이지 않다. 반면 Transformer는 "레이어를 더 쌓으면 성능이 올라간다"는 **scaling law**가 잘 성립한다 (이는 LLM에서 이미 검증된 사실이다).
> 2. **모달리티 통합**: U-Net에서 텍스트 정보는 Cross-Attention으로 "외부에서 주입"되는 형태이다. DiT에서는 텍스트 토큰과 이미지 패치 토큰을 **동일한 시퀀스에 넣고** Self-Attention으로 처리한다. 이로 인해 텍스트와 이미지가 **동등한 위치**에서 상호작용하여, 더 정밀한 텍스트-이미지 정렬이 가능하다.
> 3. **하드웨어 최적화**: 최신 GPU(A100, H100)는 Matrix Multiplication에 최적화되어 있으며, Transformer의 연산은 대부분 MatMul이다. U-Net의 Conv 연산보다 GPU 활용도가 높다.

MM-DiT의 핵심 구조는, 이미지 패치 토큰 z_{patches}와 텍스트 토큰 c를 하나의 시퀀스로 결합하여 Joint Self-Attention을 수행하는 것이다:

$$[\hat{z}_{\text{patches}}; \hat{c}] = \text{JointAttention}([z_{\text{patches}}; c])$$

단, 이미지와 텍스트는 서로 다른 modality이므로, 각각 **독립적인 MLP와 normalization**을 가진다 (이것이 "Multi-Modal"의 의미이다).

패러다임 전환 2: DDPM → Flow Matching

> 💡 **왜 DDPM에서 Flow Matching으로 바꿨는가?** DDPM은 이산 타임스텝, 복잡한 노이즈 스케줄(β_t, $\bar{\alpha}_t$ 등), 다양한 파라미터화(ϵ, v, x_0 prediction) 등 **설계 선택이 매우 많다**. 이 각각의 선택이 성능에 영향을 미치며, 최적 조합을 찾기 어렵다.

> Flow Matching은 이를 극적으로 단순화한다. 핵심 아이디어는 **데이터와 노이즈 사이의 직선 경로(straight path)**를 학습하는 것이다:
>
> $$z_t = (1-t)\cdot z_0 + t\cdot \epsilon, \quad t \in [0,1]$$
>
> 이 직선 경로(Rectified Flow)는 DDPM의 복잡한 SDE/ODE 경로보다 **더 짧고 직관적**이다. 경로가 짧으면 적은 스텝으로도 정확한 생성이 가능하다.
>
> Flow Matching의 Loss도 단순하다:
>
> $$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,z_0,\epsilon}\left[\|v_\theta(z_t,t) - (\epsilon - z_0)\|^2\right]$$
>
> 여기서 v_θ는 속도장(velocity field)을 예측하며, 타겟 $(\epsilon - z_0)$은 z_0에서 ϵ으로의 방향이다. 복잡한 노이즈 스케줄 없이 단순한 선형 보간만 사용한다.

패러다임 전환 3: 트리플 텍스트 인코더

SD 3는 CLIP ViT-L + CLIP ViT-bigG에 **T5-XXL**을 추가했다. T5-XXL은 순수 언어 모델로, CLIP과는 근본적으로 다른 텍스트 표현을 제공한다:

- **CLIP 계열**: 이미지-텍스트 유사도에 최적화 → "시각적 개념"의 표현에 강함
- **T5-XXL**: 언어 이해에 최적화 → "복잡한 문장 구조, 속성 바인딩, 공간 관계"의 이해에 강함

예를 들어, "a blue cube on top of a red sphere"에서 CLIP은 "파란색", "큐브", "빨간색", "구"가 있다는 것은 잘 포착하지만, "파란색이 큐브에, 빨간색이 구에 연결된다"는 **속성 바인딩(attribute binding)**을 혼동할 수 있다. T5-XXL은 언어 모델로서 이런 구조적 관계를 더 정확히 인코딩한다.

5.5 FLUX (2024.08, Black Forest Labs)

📄 Black Forest Labs, "FLUX.1 — Technical Report", 2024

변경	상세
Backbone	DiT 변형
텍스트 인코더	T5-XXL 단독
학습 방식	Flow Matching + Guidance Distillation
특징	프롬프트 이해도 최고 수준

> 💡 **왜 FLUX는 CLIP을 버리고 T5-XXL만 사용하는가?** SD 3에서 트리플 인코더의 성능을 분석한 결과, **T5-XXL이 텍스트 이해의 대부분을 담당**하고 있었다. CLIP 인코더를 제거해도 텍스트-이미지 정렬 품질이 크게 떨어지지 않았다. 반면 T5-XXL을 제거하면 복잡한 프롬프트에서 성능이 크게 하락했다. 따라서 FLUX는 아키텍처를 단순화하면서 가장 중요한 T5-XXL만 유지했다. 이는 "모든 것을 추가하는 것보다, 가장 효과적인 것만 남기는 것이 낫다"는 엔지니어링 원칙의 적용이다.

Guidance Distillation:

FLUX의 또 다른 혁신은 Guidance Distillation이다. 기존 CFG는 매 스텝마다 **U-Net을 2번 호출**해야 하므로 생성 시간이 2배이다. Guidance Distillation은 CFG가 적용된 교사 모델(teacher)의 출력을, CFG 없이 1회 호출만으로 재현하는 학생 모델(student)을 학습한다. 결과적으로 **CFG 품질을 유지하면서 생성 속도를 2배**로 향상시킨다.

---

6. 조건부 제어 확장

> 💡 **왜 조건부 제어가 LDM의 핵심 강점인가?** GAN 기반 모델(StyleGAN 등)은 구조적으로 조건을 추가하기 어렵다 — 조건마다 별도의 아키텍처 변경이 필요했다. 반면 LDM의 Cross-Attention 메커니즘은

> **어떤 형태의 시퀀스 데이터든** 조건으로 주입할 수 있는 **범용 인터페이스**를 제공한다. 이것이 LDM을 기반으로 ControlNet, IP-Adapter, T2I-Adapter 등 수십 가지 조건부 확장이 가능했던 이유이다.

6.1 LDM의 유연한 조건부 메커니즘

Cross-Attention은 텍스트뿐 아니라 **어떤 시퀀스든** 조건으로 사용할 수 있다:

조건 유형	인코더	적용 방식
텍스트	CLIP / T5	Cross-Attention
클래스 라벨	Embedding lookup	AdaLN / Addition
이미지 (img2img)	VAE Encoder	초기 노이즈에 혼합
레이아웃	CNN	Concatenation
깊이 맵	CNN	ControlNet
엣지	CNN	ControlNet

LDM 논문에서 제시한 일반적인 조건 주입 공식을 다시 살펴보자:

$$\mathcal{L} = \mathbb{E}_{t,z_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(y))\|^2\right]$$

여기서 y는 **임의의 조건 입력**(텍스트, 이미지, 세그먼테이션 맵, 깊이 맵 등)이고, $\tau_\theta(y)$는 조건을 인코딩하는 **도메인 특화 인코더**이다. 핵심은 U-Net의 구조를 바꾸지 않고, τ_θ만 바꾸면 **다른 종류의 조건**을 사용할 수 있다는 점이다. 이 설계가 LDM의 확장성의 근본적인 이유이다.

조건 주입 방식은 크게 세 가지이다:

1. **Cross-Attention**: 시퀀스 형태의 조건(텍스트, 토큰 리스트 등)에 적합. 각 공간 위치가 조건의 관련 부분을 선택적으로 참조한다.
2. **Concatenation**: 공간 구조가 있는 조건(깊이 맵, 엣지 맵 등)에 적합. 조건 맵을 입력 잠재 표현에 채널 방향으로 연결한다. 단, 입력 채널 수가 바뀌므로 U-Net 입력 레이어의 수정이 필요하다.
3. **Addition/AdaLN**: 전역적 조건(클래스 라벨, 타임스텝 등)에 적합. 조건 벡터를 특징 맵에 더하거나 normalization 파라미터로 주입한다.

6.2 Image-to-Image (img2img)

> 💡 **왜 img2img가 가능한가?** 디퓨전 모델의 forward process는 **어떤 시작점에서든 노이즈를 추가**할 수 있다. 순수 노이즈(z_T)에서 시작하면 완전히 새로운 이미지를 생성하고, 원본에 약간의 노이즈만 추가한 $z_{t_{\text{start}}}$에서 시작하면 원본의 구조를 유지하면서 변형할 수 있다. 이것은 디퓨전 모델의 **유연성**을 보여주는 핵심 예시이다 — GAN에서는 이런 연속적인 변형 제어가 매우 어렵다.

기존 이미지를 기반으로 변형:

```
1. 입력 이미지 → [Encoder] → z₀
2. 중간 타임스텝 t_start까지 노이즈 추가: z_{t_start}
3. z_{t_start}에서 역과정 시작 (z_T가 아닌!)
4. 텍스트 조건으로 가이드하면서 디노이징
```

- **strength** 파라미터 (0~1): 얼마나 많이 변형할지
  - 0.3: 원본 유지하면서 약간 변형
  - 0.8: 구도만 유지, 내용은 크게 변형
  - 1.0: 완전히 새로 생성 (txt2img와 동일)

strength와 타임스텝의 수학적 관계:

strength s가 주어지면, 시작 타임스텝은 $t_{\text{start}} = \lfloor s \times T \rfloor$이다. 이 시점에서의 노이즈 수준을 보면:

$$z_{t_{\text{start}}} = \sqrt{\bar{\alpha}_{t_{\text{start}}}} \cdot z_0 + \sqrt{1 - \bar{\alpha}_{t_{\text{start}}}} \cdot \epsilon$$

- $s = 0.3$일 때: $t_{\text{start}} = 300$, $\bar{\alpha}_{300} \approx 0.65$ → 원본 신호가 $\sqrt{0.65} \approx 81\%$ 유지
- $s = 0.8$일 때: $t_{\text{start}} = 800$, $\bar{\alpha}_{800} \approx 0.02$ → 원본 신호가 $\sqrt{0.02} \approx 14\%$만 유지, 거의 노이즈

따라서 strength는 "원본 정보를 얼마나 파괴할 것인가"를 직접 제어한다. 디노이징 과정에서 텍스트 프롬프트의 영향은 파괴된 정보를 새로운 내용으로 채우는 역할을 한다.

6.3 Inpainting

> 💡 **왜 Inpainting이 디퓨전에서 자연스럽게 작동하는가?** Inpainting은 "이미지의 일부분만 새로 생성"하는 것이다. GAN에서는 이를 위해 별도의 마스크 처리 아키텍처가 필요하지만, 디퓨전에서는 **매 스텝마다 마스크를 적용하는 것만으로** 구현된다. 이것이 가능한 이유는 디퓨전의 각 스텝이 **독립적인 로컬 업데이트**이기 때문이다.

마스크 영역만 새로 생성:

```
z_t에서 매 디노이징 스텝마다:
  z_t = mask · z_t_denoised + (1-mask) · z_t_original
```

마스크 외 영역은 원본에서 가져오고, 마스크 내부만 디퓨전이 채운다.

이 방식의 구체적인 동작을 살펴보자:

1. 원본 이미지를 인코딩: $z_0^{\text{orig}} = E(x_{\text{orig}})$
2. 매 타임스텝 t에서, 원본의 해당 노이즈 수준 버전을 계산: $z_t^{\text{orig}} = \sqrt{\bar{\alpha}_t} z_0^{\text{orig}} + \sqrt{1 - \bar{\alpha}_t} \epsilon$
3. 디노이징된 결과와 원본을 마스크로 합성: $z_t = m \odot z_t^{\text{denoised}} + (1 - m) \odot z_t^{\text{orig}}$

여기서 m은 마스크(생성할 영역 = 1, 유지할 영역 = 0), $\odot$는 element-wise 곱이다.

단, 이 "naive inpainting"은 마스크 경계에서 **불연속**이 발생할 수 있다. 마스크 내부와 외부가 서로 다른 맥락에서 생성되기 때문이다. 이를 해결하기 위해 SD inpainting 전용 모델은 마스크를 U-Net의 **추가 입력 채널**로 받아, 경계를 인식하면서 자연스러운 전환을 생성한다.

6.4 PyTorch 구현: Image-to-Image (img2img)

```python
import torch
import torch.nn.functional as F
import numpy as np
from PIL import Image


class Img2ImgPipeline:
    """
    Image-to-Image 파이프라인

    기존 이미지를 기반으로, 텍스트 프롬프트에 따라 변형하는 기법.
    핵심 아이디어: 완전한 노이즈(z_T)에서 시작하지 않고,
    원본 이미지에 부분적으로 노이즈를 추가한 중간 지점(z_{t_start})에서 시작.
    """
    def __init__(self, vae, unet, text_encoder, device='cuda'):
        self.vae = vae.to(device).eval()
        self.unet = unet.to(device).eval()
        self.text_encoder = text_encoder.to(device).eval()
        self.device = device

        # 노이즈 스케줄
        betas = torch.linspace(0.00085 ** 0.5, 0.012 ** 0.5, 1000) ** 2
        alphas = 1.0 - betas
        self.alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
        self.num_timesteps = 1000
        self.scaling_factor = 0.18215

    @torch.no_grad()
    def generate(
        self,
        init_image,                 # 원본 이미지 텐서 [1, 3, H, W]
        prompt_tokens,              # 텍스트 프롬프트 토큰
        strength=0.75,              # 변형 강도 (0=원본 유지, 1=완전 재생성)
        num_inference_steps=50,
        guidance_scale=7.5,
        seed=42
    ):
        """
        img2img 생성

        strength에 따라 시작 타임스텝이 결정됨:
        - strength=0.75 → 전체 50 스텝 중 37 스텝부터 디노이징 시작
        - strength=0.3  → 15 스텝부터 시작 (원본에 가까움)
        """
        generator = torch.Generator(device=self.device).manual_seed(seed)

        # === 1단계: 원본 이미지를 잠재 공간으로 인코딩 ===
        z_0, _, _ = self.vae.encode(init_image.to(self.device))

        # === 2단계: 시작 타임스텝 계산 ===
        # strength가 높을수록 더 많은 노이즈 → 더 큰 변형
        t_start = int(self.num_timesteps * strength)
        print(f"  변형 강도: {strength} → 타임스텝 {t_start}에서 시작")

        # === 3단계: 원본에 노이즈 추가 (forward process) ===
        noise = torch.randn_like(z_0, generator=generator)
        alpha_t = self.alphas_cumprod[t_start]

        # z_{t_start} = √ᾱ_{t_start} · z₀ + √(1 - ᾱ_{t_start}) · ε
        z_t = torch.sqrt(alpha_t) * z_0 + torch.sqrt(1 - alpha_t) * noise

        # === 4단계: 텍스트 조건 인코딩 ===
```

```
        c = self.text_encoder(prompt_tokens.to(self.device))
        uncond_tokens = torch.zeros_like(prompt_tokens)
        uc = self.text_encoder(uncond_tokens.to(self.device))

        # === 5단계: 디노이징 (t_start부터 0까지만) ===
        # 전체 스텝이 아닌 일부만 수행 → 원본 구조 보존
        num_denoise_steps = int(num_inference_steps * strength)
        step_ratio = t_start // max(num_denoise_steps, 1)
        timesteps = torch.arange(num_denoise_steps, device=self.device) * step_ratio
        timesteps = timesteps.flip(0)

        for i, t in enumerate(timesteps):
            t_batch = t.unsqueeze(0)

            # CFG 적용
            noise_uncond = self.unet(z_t, t_batch, uc)
            noise_cond = self.unet(z_t, t_batch, c)
            noise_pred = noise_uncond + guidance_scale * (noise_cond - noise_uncond)

            # DDIM 업데이트
            alpha_t_cur = self.alphas_cumprod[t.long()]
            if i + 1 < len(timesteps):
                alpha_prev = self.alphas_cumprod[timesteps[i + 1].long()]
            else:
                alpha_prev = torch.tensor(1.0, device=self.device)

            z_0_pred = (z_t - torch.sqrt(1 - alpha_t_cur) * noise_pred) / torch.sqrt(
            z_t = torch.sqrt(alpha_prev) * z_0_pred + torch.sqrt(1 - alpha_prev) * no

        # === 6단계: 디코딩 ===
        image = self.vae.decode(z_t)
        image = (image.clamp(-1, 1) + 1) / 2 * 255
        image = image[0].permute(1, 2, 0).cpu().numpy().astype(np.uint8)
        return Image.fromarray(image)


# === 사용 예시 ===
# pipeline = Img2ImgPipeline(vae, unet, text_encoder)
#
# # 원본 이미지 로드 (전처리: [0,255] → [-1,1])
# init_image = load_and_preprocess("input_photo.png")  # [1, 3, 256, 256]
#
# # strength별 비교:
# for s in [0.3, 0.5, 0.75, 1.0]:
#     result = pipeline.generate(
#         init_image=init_image,
#         prompt_tokens=tokenizer("a watercolor painting of a landscape"),
#         strength=s,
#         guidance_scale=7.5
#     )
#     result.save(f"img2img_strength_{s}.png")
#     print(f"strength={s}: 저장 완료")
```

---

7. 실용적 고려사항

7.1 샘플러 선택

> 💡 **왜 샘플러 선택이 중요한가?** 학습된 디퓨전 모델은 하나이지만, 생성 시 사용하는 샘플러에 따라 **품질, 속도, 다양성**이 크게 달라진다. 같은 모델이라도 DDPM으로 1000스텝 생성한 결과와 DPM-

> Solver++로 20스텝 생성한 결과가 비슷한 품질을 보일 수 있다. 이는 샘플러가 풀고 있는 미분 방정식(SDE/ODE)을 **더 효율적으로 수치 적분**하기 때문이다.

샘플러	스텝 수	유형	용도
DDPM	1000	확률적	이론적 참조
DDIM	20~50	결정론적	빠른 생성, 일관성
Euler	20~30	ODE	단순, 범용
DPM-Solver++	15~25	고차 ODE	빠르고 고품질
Euler Ancestral	20~30	확률적	다양성 높음

확률적 vs 결정론적 샘플러의 차이:

- **결정론적(Deterministic)** 샘플러(DDIM, Euler, DPM-Solver++): 동일한 초기 노이즈 z_T와 동일한 프롬프트에서 항상 **동일한 이미지**를 생성한다. 이는 역과정을 ODE(Ordinary Differential Equation)로 모델링하기 때문이다. 시드 기반 재현성이 완벽하여 디버깅과 비교 실험에 유리하다.
- **확률적(Stochastic)** 샘플러(DDPM, Euler Ancestral): 각 스텝에서 **추가 노이즈를 주입**한다. 이는 SDE(Stochastic Differential Equation)를 따르며, 동일 시드에서도 약간 다른 결과를 만든다. 일반적으로 **다양성이 높고** 아티팩트가 적지만, 수렴에 더 많은 스텝이 필요할 수 있다.

고차 솔버가 빠른 이유: DPM-Solver++같은 고차 ODE 솔버는 이전 여러 스텝의 정보를 활용하여 현재 스텝의 업데이트를 더 정확하게 추정한다. 수치 해석에서 Euler 방법(1차)이 한 스텝에 $O(h)$ 오차를 가지는 반면, 2차/3차 방법은 $O(h^2)$, $O(h^3)$ 오차를 가진다. 같은 품질을 위해 더 큰 스텝 크기(h)를 사용할 수 있으므로, 총 스텝 수가 줄어든다.

7.2 메모리 최적화

> 💡 **왜 메모리 최적화가 필요한가?** SD 1.x의 U-Net만 ~3.4GB(fp16), VAE ~160MB, CLIP ~500MB로, 총 ~4GB 이상의 GPU 메모리가 필요하다. CFG를 사용하면 배치 크기가 2배가 되어 중간 활성값(activation) 메모리도 2배이다. 512×512 생성에 약 6-8GB, 1024×1024에는 12GB 이상이 필요하다. 소비자 GPU에서 이를 실행하려면 최적화가 필수적이다.

- **Attention Slicing**: Attention을 청크로 분할하여 순차 계산 → 메모리 절약
- **VAE Tiling**: 큰 이미지를 타일로 나누어 디코딩
- **xFormers**: 메모리 효율적 Attention 구현
- **Half Precision (fp16)**: 메모리 절반, 속도 향상

각 최적화 기법의 원리를 좀 더 살펴보자:

- **Attention Slicing**: Self-Attention의 QK^T 행렬은 시퀀스 길이의 제곱에 비례하는 메모리를 차지한다. 예를 들어 32×32=1024 토큰의 Self-Attention은 1024×1024 = 1M 원소의 행렬이 필요하다. Slicing은 이 행렬을 여러 조각으로 나누어 순차적으로 계산함으로써 피크 메모리를 줄인다. 속도는 약간 느려지지만, 메모리는 크게 절약된다.
- **xFormers / Flash Attention**: Attention 계산을 수학적으로 재배열하여, QK^T 전체 행렬을 메모리에 올리지 않고 **블록 단위로 계산 → 즉시 softmax → 즉시 V 곱**을 수행한다. 이론적으로 메모리 복잡도가 $O(N^2)$에서 $O(N)$으로 줄어들며, GPU 캐시 활용이 최적화되어 **속도도 향상**된다.
- **VAE Tiling**: VAE 디코더는 전체 잠재 표현을 한 번에 디코딩한다. 고해상도(예: 2048×2048)에서는 디코더의 중간 활성값이 매우 커진다. Tiling은 잠재 표현을 겹치는 타일로 분할하여 각각 디코딩한 뒤, 겹치는 부분을 블렌딩하여 합성한다.

7.3 Fine-tuning 방법

> 💡 **왜 Fine-tuning이 필요한가?** 사전학습된 SD는 수억 장의 일반적인 이미지로 학습되었다. 특정 스타일(지브리 애니메이션), 특정 대상(나의 반려동물), 특정 도메인(의료 영상)에 대해서는 해당 데이터가 학

> 습 셋에 충분하지 않아 품질이 떨어진다. Fine-tuning은 소량의 타겟 데이터로 모델을 **특화**시키는 과정이다. 문제는 전체 모델을 fine-tuning하면 학습 비용이 크고, 기존 능력을 잊을 수 있다는 것이다 (catastrophic forgetting). 이를 해결하는 다양한 효율적 방법이 개발되었다.

방법	학습 파라미터	용도
Full Fine-tuning	전체 U-Net	도메인 특화 (의료, 위성 등)
LoRA	저랭크 어댑터 (~4M)	스타일, 캐릭터 학습
Textual Inversion	텍스트 임베딩 1개	특정 개념 학습
DreamBooth	U-Net + 텍스트	특정 대상 (나의 얼굴 등)

각 방법의 차이를 직관적으로 이해하자:

- **Textual Inversion**: "이 개념을 설명하는 새로운 단어를 만드는 것". 모델의 가중치는 전혀 바꾸지 않고, CLIP 토큰 임베딩 공간에서 새로운 벡터 v_*를 최적화한다. 학습 가능한 파라미터가 단 하나의 임베딩 벡터(768차원)뿐이므로 극도로 가볍지만, 표현력도 제한적이다.
- **DreamBooth**: "모델에게 새로운 개념을 깊이 가르치는 것". U-Net 전체를 fine-tuning하되, 기존 지식을 잊지 않도록 **Prior Preservation Loss**를 추가한다. 가장 높은 품질을 달성하지만, 학습 비용이 크고 모델 전체를 저장해야 한다(~3.5GB).
- **LoRA**: "모델의 핵심 능력에 작은 조정을 가하는 것". Textual Inversion보다 표현력이 높고, DreamBooth보다 효율적인 **황금 균형점**이다.

7.4 PyTorch 구현: 간소화된 LoRA Fine-tuning

📄 Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022

⚠️ **LoRA의 핵심 아이디어**: 사전학습된 가중치 W_0를 고정하고, 저랭크 행렬 $\Delta W = BA$만 학습한다. $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, 여기서 랭크 $r \ll \min(d, k)$.

$$W = W_0 + \Delta W = W_0 + BA$$

> 💡 **왜 저랭크 근사가 작동하는가?** Aghajanyan et al. (2020)은 사전학습된 모델의 가중치 업데이트 ΔW가 실제로 **매우 낮은 내재 랭크(intrinsic rank)**를 가진다는 것을 실험적으로 보였다. 직관적으로, fine-tuning은 사전학습된 모델의 "거대한 지식"을 약간만 조정하는 것이므로, 변화량 ΔW가 전체 가중치 행렬에 비해 매우 단순한 구조를 가진다. 랭크 $r = 4$이면 ΔW의 열 공간이 4차원이라는 뜻으로, "4개의 독립적인 방향으로만 가중치를 조정"하는 것과 같다. 놀랍게도 이 4차원 조정만으로 스타일, 캐릭터 등을 효과적으로 학습할 수 있다.
>
> **초기화의 중요성**: B를 0으로 초기화하면, 학습 시작 시 $\Delta W = BA = 0$이므로 원래 모델과 동일하게 시작한다. 이는 학습 초기에 모델이 "갑자기 나빠지는" 문제를 방지하며, **점진적인 적응**을 가능하게 한다. A는 Kaiming 초기화로 다양한 방향을 탐색할 수 있게 한다.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Optional


class LoRALinear(nn.Module):
    """
    LoRA (Low-Rank Adaptation) 레이어

    원래 Linear 레이어의 가중치를 고정하고,
    저랭크 분해된 업데이트 행렬 ΔW = B × A만 학습.

    파라미터 효율성:
    - 원래: d × k 파라미터 (예: 1024 × 1024 = 1,048,576)
    - LoRA: d × r + r × k (예: r=4이면 1024×4 + 4×1024 = 8,192)
    - 절감률: 99.2%
    """
    def __init__(
        self,
        original_layer: nn.Linear,
        rank: int = 4,
        alpha: float = 1.0,
    ):
        super().__init__()
        self.original_layer = original_layer
        self.rank = rank
        self.alpha = alpha
        self.scaling = alpha / rank  # LoRA 스케일링 팩터

        d_out, d_in = original_layer.weight.shape

        # 원래 가중치 고정
        original_layer.weight.requires_grad_(False)
        if original_layer.bias is not None:
            original_layer.bias.requires_grad_(False)

        # LoRA 행렬: ΔW = B × A
        # A: 입력 → 저랭크 (Kaiming 초기화)
        self.lora_A = nn.Parameter(torch.randn(rank, d_in) * (1.0 / rank))
        # B: 저랭크 → 출력 (0으로 초기화 → 초기에는 ΔW = 0)
        self.lora_B = nn.Parameter(torch.zeros(d_out, rank))

    def forward(self, x):
        # 원래 레이어 출력 (고정)
        original_out = self.original_layer(x)

        # LoRA 업데이트: x @ A^T @ B^T * scaling
        lora_out = F.linear(F.linear(x, self.lora_A), self.lora_B)
        lora_out = lora_out * self.scaling

        return original_out + lora_out


def apply_lora_to_unet(unet, rank=4, alpha=1.0, target_modules=None):
    """
    U-Net의 Attention 레이어에 LoRA를 적용

    Args:
        unet: 사전학습된 U-Net 모델
        rank: LoRA 랭크 (낮을수록 파라미터 적음, 보통 4~16)
        alpha: LoRA 스케일링 팩터
```

```
        target_modules: LoRA를 적용할 모듈 이름 리스트
    """
    if target_modules is None:
        # 기본: Cross-Attention의 Q, K, V, Out 프로젝션에 적용
        target_modules = ['to_q', 'to_k', 'to_v', 'to_out']

    lora_params = []
    replaced_count = 0

    for name, module in unet.named_modules():
        # 타겟 모듈 이름이 포함된 Linear 레이어 찾기
        for target in target_modules:
            if target in name and isinstance(module, nn.Linear):
                # 원래 Linear를 LoRA 래퍼로 교체
                parent_name = '.'.join(name.split('.')[:-1])
                child_name = name.split('.')[-1]
                parent = dict(unet.named_modules())[parent_name]

                lora_layer = LoRALinear(module, rank=rank, alpha=alpha)
                setattr(parent, child_name, lora_layer)

                # LoRA 파라미터만 수집
                lora_params.extend([lora_layer.lora_A, lora_layer.lora_B])
                replaced_count += 1

    # 원래 U-Net 파라미터는 모두 고정
    for name, param in unet.named_parameters():
        if 'lora_' not in name:
            param.requires_grad_(False)

    # 학습 가능한 파라미터 수 비교
    total_params = sum(p.numel() for p in unet.parameters())
    trainable_params = sum(p.numel() for p in unet.parameters() if p.requires_grad)
    print(f"LoRA 적용 완료:")
    print(f"  교체된 레이어: {replaced_count}개")
    print(f"  전체 파라미터:     {total_params:>12,}")
    print(f"  학습 파라미터:     {trainable_params:>12,} ({trainable_params/total_para

    return lora_params


class LoRAFineTuner:
    """
    LoRA Fine-tuning 학습기

    사용 사례:
    - 특정 아트 스타일 학습 (예: 지브리, 수채화)
    - 특정 캐릭터/대상 학습 (DreamBooth와 유사)
    - 도메인 적응 (의료 영상, 위성 사진 등)
    """
    def __init__(self, vae, unet, text_encoder, lora_params, device='cuda'):
        self.vae = vae.to(device).eval()
        self.unet = unet.to(device)
        self.text_encoder = text_encoder.to(device).eval()
        self.device = device

        # LoRA 파라미터만 학습
        self.optimizer = torch.optim.AdamW(lora_params, lr=1e-4, weight_decay=0.01)

        # 노이즈 스케줄
        betas = torch.linspace(0.00085 ** 0.5, 0.012 ** 0.5, 1000) ** 2
        alphas = 1.0 - betas
```

```
        self.alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
        self.num_timesteps = 1000

    def train_step(self, images, text_tokens):
        """LoRA fine-tuning 학습 스텝 (LDM 학습과 동일한 구조)"""
        batch_size = images.shape[0]

        # 이미지 → 잠재 공간
        with torch.no_grad():
            z_0, _, _ = self.vae.encode(images.to(self.device))
            c = self.text_encoder(text_tokens.to(self.device))

        # 랜덤 타임스텝 + 노이즈
        t = torch.randint(0, self.num_timesteps, (batch_size,), device=self.device)
        noise = torch.randn_like(z_0)
        alpha_t = self.alphas_cumprod[t][:, None, None, None]
        z_t = torch.sqrt(alpha_t) * z_0 + torch.sqrt(1 - alpha_t) * noise

        # 노이즈 예측 (LoRA가 적용된 U-Net)
        noise_pred = self.unet(z_t, t, c)

        # MSE Loss + 역전파 (LoRA 파라미터만 업데이트)
        loss = F.mse_loss(noise_pred, noise)
        self.optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(
            [p for p in self.unet.parameters() if p.requires_grad],
            max_norm=1.0
        )
        self.optimizer.step()

        return loss.item()

    def save_lora_weights(self, path):
        """LoRA 가중치만 저장 (매우 작은 파일 크기)"""
        lora_state = {}
        for name, param in self.unet.named_parameters():
            if 'lora_' in name:
                lora_state[name] = param.data.clone()
        torch.save(lora_state, path)
        print(f"LoRA 가중치 저장: {path}")
        size_mb = sum(v.numel() * 4 for v in lora_state.values()) / 1024 / 1024
        print(f"  파일 크기: {size_mb:.1f} MB (전체 모델 대비 매우 작음)")


# === 사용 예시 ===
if __name__ == "__main__":
    print("=== LoRA Fine-tuning 워크플로우 ===")
    print()
    print("1. 사전학습된 SD 모델 로드")
    print("2. U-Net의 Attention 레이어에 LoRA 적용 (rank=4)")
    print("3. 타겟 이미지-텍스트 쌍으로 학습 (보통 수백~수천 장)")
    print("4. LoRA 가중치만 저장 (~4MB)")
    print()
    print("장점:")
    print("  - 학습 파라미터: 전체의 ~0.1% → GPU 메모리 절약")
    print("  - 학습 시간: 수 시간 (Full fine-tune 대비 수십 배 빠름)")
    print("  - 저장 용량: ~4MB (전체 모델 ~3.5GB 대비)")
    print("  - 여러 LoRA를 조합하여 스타일 혼합 가능")
```

8. 핵심 수식 요약

이름	수식
인코딩	$z_0 = E(x_0) \cdot s$ (scaling factor s)
Forward	$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$
학습 Loss	$\mathcal{L} = \mathbb{E}_{t,z_0,\epsilon}\left[\lvert\epsilon - \epsilon_\theta(z_t, t, c)\rvert^2\right]$
CFG	$\tilde{\epsilon} = \epsilon_\theta(z_t, \varnothing) + w(\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing))$
디코딩	$x_0 = D(z_0/s)$
LoRA 업데이트	$W = W_0 + BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k}$
img2img 시작점	$z_{t_{\text{start}}} = \sqrt{\bar{\alpha}_{t_{\text{start}}}}\, z_0 + \sqrt{1-\bar{\alpha}_{t_{\text{start}}}}\, \epsilon$
v-prediction	$v_t = \sqrt{\bar{\alpha}_t}\, \epsilon - \sqrt{1-\bar{\alpha}_t}\, x_0$
Cross-Attention	$\text{Attn}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$, 여기서 $Q = \phi W_Q$, $K = \tau_\theta(c) W_K$
AE Loss	$\mathcal{L}_{\text{AE}} = \lvert x - D(E(x))\rvert^2 + \lambda_{\text{KL}} D_{\text{KL}} + \lambda_{\text{LPIPS}}\text{LPIPS} + \lambda_{\text{GAN}}\mathcal{L}_{\text{GAN}}$
Flow Matching	$z_t = (1-t)z_0 + t\epsilon$, $\mathcal{L}_{\text{FM}} = \lvert v_\theta(z_t, t) - (\epsilon - z_0)\rvert^2$
SNR	$\text{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$

추가 학습 기법 수식:

Noise Offset — 매우 밝거나 어두운 이미지 생성을 가능하게 하는 기법:

$$\epsilon_{\text{offset}} = \epsilon + \delta \cdot \mathcal{N}(0,1) \cdot \mathbf{1}$$

여기서 $\delta \approx 0.1$이고, $\mathbf{1}$은 모든 공간 위치에서 동일한 값을 가지는 벡터(= 이미지의 평균 밝기에만 영향)이다.

> 💡 **왜 Noise Offset이 필요한가?** 표준 DDPM에서 $\epsilon \sim \mathcal{N}(0, I)$은 **각 픽셀이 독립**이므로, 공간 평균 $\frac{1}{HW}\sum \epsilon_{ij} \approx 0$이 된다 (큰 수의 법칙). 이는 노이즈가 추가되어도 **이미지의 전체 평균 밝기가 거의 변하지 않는다**는 뜻이다. 결과적으로 모델은 학습 데이터의 평균 밝기에서 크게 벗어나는 이미지(매우 어두운 야경, 매우 밝은 설경)를 생성하기 어렵다. Noise Offset은 **공간적으로 상관된 노이즈 성분**(= 전체 밝기 이동)을 추가하여, 모델이 다양한 평균 밝기를 학습할 수 있게 한다.

Min-SNR Weighting — 타임스텝별 Loss 가중치를 조정하는 기법:

$$\mathcal{L}_{\text{Min-SNR}} = \mathbb{E}_t\left[\min\left(\text{SNR}(t), \gamma\right) \cdot \lVert\epsilon - \epsilon_\theta(z_t, t)\rVert^2\right]$$

여기서 $\text{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$이고, γ는 클램핑 상수(보통 5)이다.

> 💡 **왜 Min-SNR Weighting이 필요한가?** $\text{SNR}(t)$는 타임스텝 t에서의 **신호 대 잡음비**이다:
>
> - t 작음 (노이즈 적음): SNR 매우 큼 (수천~무한) → gradient가 매우 큰 가중치를 받아 **폭발**
> - t 큼 (노이즈 많음): SNR 매우 작음 (0에 가까움) → gradient 기여가 매우 작음
>
> 이 불균형 때문에 표준 학습에서는 낮은 t (적은 노이즈, 세부 디테일 학습)가 Loss를 지배하고, 높은 t (많은 노이즈, 전체 구조 학습)는 무시된다. Min-SNR은 SNR이 γ를 초과하면 γ로 클램핑하여, **모든 타임스텝에서 균일한 gradient 기여**를 보장한다. 이는 학습 속도와 최종 품질 모두를 향상시킨다.

9. 연결 지도: 강의 1~9의 통합

> 💡 **왜 연결 지도가 중요한가?** LDM을 완전히 이해하려면, 각 구성 요소가 "이전 강의에서 배운 어떤 개념의 직접적인 적용"인지를 파악해야 한다. 이 연결 지도를 내면화하면, 새로운 모델을 접했을 때 "이것은 기존의 어떤 구성 요소를 어떻게 변형한 것인가?"라는 프레임워크로 빠르게 이해할 수 있다.

```
[강의 1: VAE]
  ELBO, 잠재 공간, Reparameterization
    ↳ 오토인코더의 잠재 공간 개념 → LDM의 Encoder/Decoder
    ↳ Reparameterization Trick → KL-VAE의 z = μ + σ·ε 샘플링

[강의 2: VQ-VAE, Normalizing Flow]
  이산 잠재 공간, Change of Variables
    ↳ VQ-GAN의 코드북 → LDM의 오토인코더 변형
    ↳ 이산 vs 연속 잠재 공간의 트레이드오프 이해 → KL-VAE 선택의 근거

[강의 3~4: GAN, StyleGAN, VQ-GAN]
  적대적 학습, 2-Stage 생성
    ↳ 오토인코더의 GAN Loss → LDM의 Stage 1
    ↳ PatchGAN 판별자 → 오토인코더 복원의 날카로움
    ↳ 2-Stage 생성 패러다임 → LDM의 "압축 → 생성" 구조

[강의 5: DALL-E, Score-Based]
  텍스트→이미지 자기회귀, 스코어 함수
    ↳ 텍스트 조건부 생성의 시작, Score = 노이즈 예측
    ↳ ▽_x log p(x) ≈ -ε_θ/√(1-ᾱ_t) 관계 → 디노이징 = 스코어 추정

[강의 7: DDPM]
  Forward/Reverse, Simple Loss, U-Net
    ↳ LDM의 Stage 2 (잠재 공간에서 동일한 DDPM)
    ↳ 노이즈 스케줄, ᾱ_t 체계 → LDM에서 동일하게 사용
    ↳ U-Net 아키텍처 → SD 1.x/2.x의 backbone

[강의 8: CLIP, CFG]
  텍스트-이미지 매핑, 조건 강화
    ↳ LDM의 텍스트 조건 인코더 + 생성 품질 향상
    ↳ CLIP의 대조 학습 → 텍스트 임베딩이 시각적 개념과 정렬
    ↳ CFG의 ε˜= ε_uncond + w·(ε_cond - ε_uncond) → 생성 시 품질 제어

                ↓ ↓ ↓ 모든 것이 합쳐진다 ↓ ↓ ↓

[강의 9: Latent Diffusion / Stable Diffusion] ← 여기
  VQ-GAN Encoder → 잠재 공간 → DDPM + CLIP + CFG → VQ-GAN Decoder
```

각 강의에서 배운 개념이 LDM의 **어떤 코드 위치**에 대응하는지를 명확히 하면:

강의	개념	LDM에서의 위치
강의 1	Reparameterization Trick	`z = mean + std * eps` (VAE 인코더)
강의 4	Perceptual Loss + GAN Loss	오토인코더 학습의 Loss 함수
강의 7	Forward Process $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$	디퓨전 학습 루프의 노이즈 추가
강의 7	Simple Loss $\lvert\epsilon - \epsilon_\theta\rvert^2$	디퓨전 학습 루프의 Loss 계산

강의 8	CLIP Text Encoder	`c = text_encoder(prompt_tokens)`
강의 8	CFG	`noise_pred = noise_uncond + w * (noise_cond - noise_uncond)`

10. 다음 강의 미리보기

Stable Diffusion 1~2는 **U-Net** 기반이었지만, SD 3과 FLUX는 **DiT(Diffusion Transformer)** 기반이다. 또한 DDPM 대신 **Flow Matching**을 사용한다.

이 강의에서 다룬 LDM의 핵심 프레임워크 — "잠재 공간에서의 조건부 생성" — 는 다음 강의에서도 **동일하게 유지**된다. 바뀌는 것은 "잠재 공간에서 어떤 방식으로 생성하는가"이다:

- **강의 10**: Flow Matching — ODE 기반 생성, 직선 경로, Optimal Transport
  - 핵심 질문: "DDPM의 복잡한 SDE 경로 대신, 데이터와 노이즈를 직선으로 연결하면 어떨까?"
  - $z_t = (1-t)z_0 + t\epsilon$이라는 단순한 보간으로 DDPM과 동등 이상의 성능을 달성
  - SD 3, FLUX의 기반 기술
- **강의 11**: DiT & 가속 샘플링 — Transformer 아키텍처, Consistency Model
  - 핵심 질문: "U-Net 대신 Transformer를 사용하면 scaling law의 이점을 얻을 수 있지 않을까?" / "50스텝이 아니라 1-4스텝으로 생성할 수 없을까?"
  - DiT: Vision Transformer를 디퓨전에 적용, 파라미터 scaling에 따른 예측 가능한 성능 향상
  - Consistency Model: 다단계 디노이징을 1-2스텝으로 증류(distillation)

참고문헌 (References)

핵심 논문

1. **Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B.** (2022). "High-Resolution Image Synthesis with Latent Diffusion Models." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684-10695. arXiv:2112.10752
2. **Esser, P., Kulal, S., Blattmann, A., Entezari, R., Muller, J., Saini, H., ... & Rombach, R.** (2024). "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis." *International Conference on Machine Learning (ICML)*. arXiv:2403.03206
3. **Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Muller, J., Penna, J., & Rombach, R.** (2023). "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis." arXiv:2307.01952

구성 요소 관련 논문

4. **Esser, P., Rombach, R., & Ommer, B.** (2021). "Taming Transformers for High-Resolution Image Synthesis." *CVPR 2021*. arXiv:2012.09841 — **VQ-GAN**
5. **Ho, J., Jain, A., & Abbeel, P.** (2020). "Denoising Diffusion Probabilistic Models." *NeurIPS 2020*. arXiv:2006.11239 — **DDPM**
6. **Radford, A., Kim, J. W., Hallacy, C., et al.** (2021). "Learning Transferable Visual Models From Natural Language Supervision." *ICML 2021*. arXiv:2103.00020 — **CLIP**
7. **Ho, J. & Salimans, T.** (2022). "Classifier-Free Diffusion Guidance." *NeurIPS 2021 Workshop*. arXiv:2207.12598 — **CFG**

8. **Song, J., Meng, C., & Ermon, S.** (2021). "Denoising Diffusion Implicit Models." *ICLR 2021*. arXiv:2010.02502 — **DDIM**

Fine-tuning 관련 논문

9. **Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W.** (2022). "LoRA: Low-Rank Adaptation of Large Language Models." *ICLR 2022*. arXiv:2106.09685 — **LoRA**
10. **Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., & Aberman, K.** (2023). "DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation." *CVPR 2023*. arXiv:2208.12242 — **DreamBooth**
11. **Gal, R., Alaluf, Y., Atzmon, Y., Patashnik, O., Bermano, A. H., Chechik, G., & Cohen-Or, D.** (2023). "An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion." *ICLR 2023*. arXiv:2208.01618 — **Textual Inversion**

ControlNet 및 확장

12. **Zhang, L., Rao, A., & Agrawala, M.** (2023). "Adding Conditional Control to Text-to-Image Diffusion Models." *ICCV 2023*. arXiv:2302.05543 — **ControlNet**